

MASTER'S THESIS · COMPUTER VISION

Real-Time Object Detection and Tracking

AC-MOT

Adaptive Control for Real-Time Multi-Object Tracking

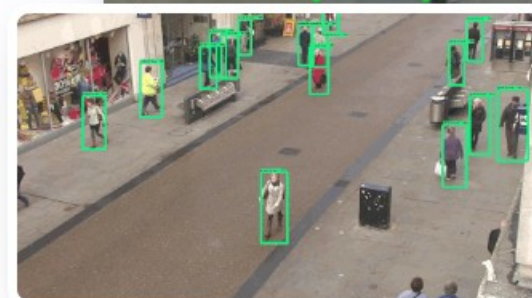
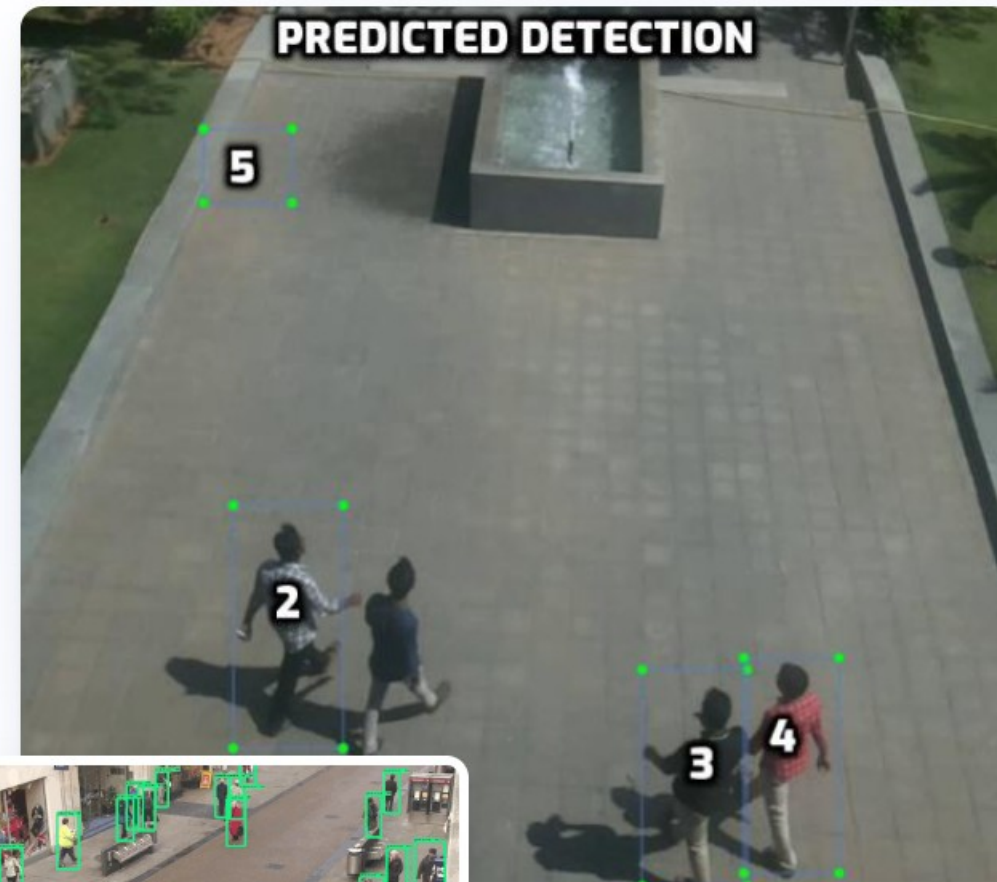
BY

**Capt. Eng. Ahmed Gouda
Ismail**

UNDER SUPERVISION OF

**Dr. Tarek Ahmed
Mahmoud
Dr. Mohamed S. Mohamed**

Military Technical College · Electrical Engineering Branch
Computer Engineering and Artificial Intelligence Department



Detect Track Adapt

Outline

I	Introduction and Motivation Detection, tracking and how we measure them slides 3 - 16	VI	V1: Validation-Driven Optimization Pre-Optuna screening, full-pipeline measurement, Optuna and Trial 24 slides 60 - 76
II	Related Work: Detectors and Trackers Detectors through the ages, trackers and benchmarks slides 17 - 30	VII	V2: Multi-Objective Optimization Pareto search and the V1 / V2 trade-off slides 77 - 84
III	Problem and Baseline Easy and hard frames, one fixed setting slides 31 - 35	VIII	Statistical and External Validation Paired bootstrap and UAVDT with zero tuning slides 85 - 90
IV	Proposed Framework: AC-MOT Measure the scene, score it, set the detector slides 36 - 50	IX	Transfer to Published U2MOT Reproduction, transfer, freeze and final test slides 91 - 97
V	Experimental Setup and First Result VisDrone, protocol and the ablation study slides 51 - 59	X	Conclusion and Future Work What we delivered and what comes next slides 98 - 102

SECTION I

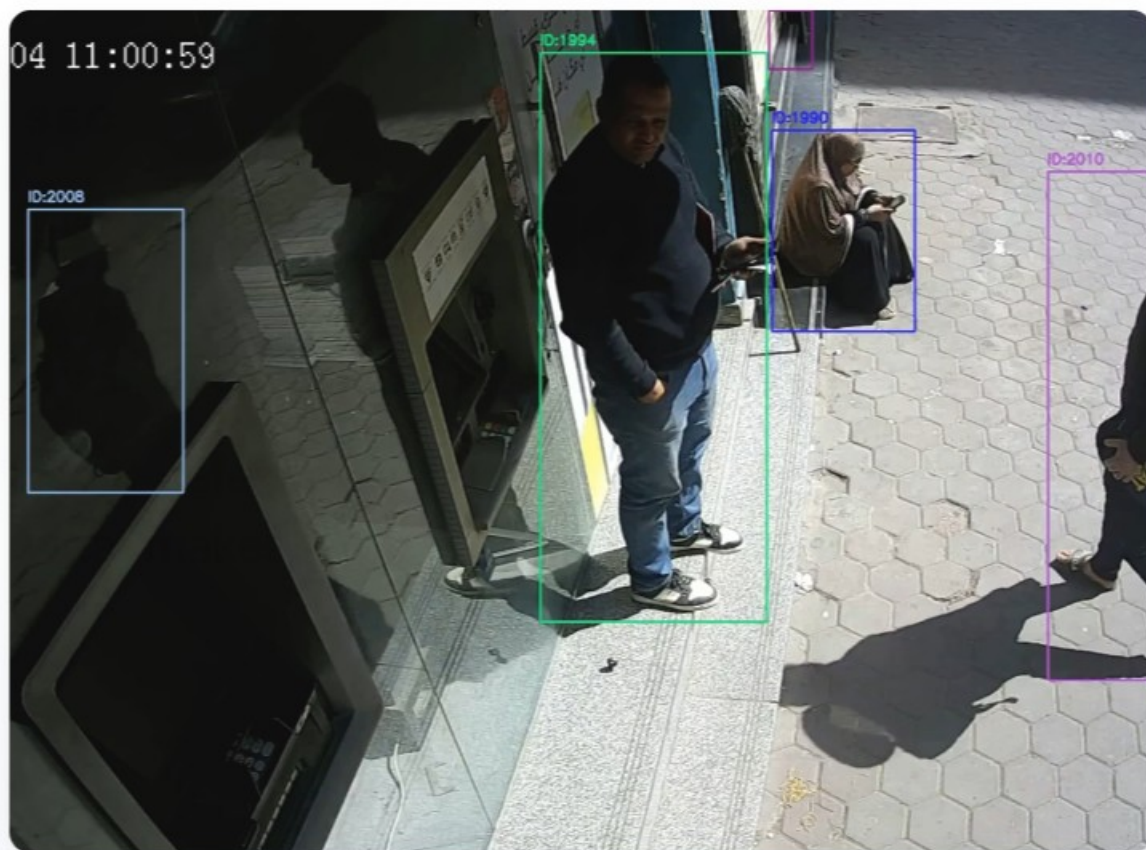
Introduction and Motivation

What detection and tracking are, where they are used, what makes them hard, and how we measure them.

Object Detection: What It Gives Us

WHAT IT MEANS

A detector looks at **one picture** and draws a **box** around each object. For each box it says **what** the object is and **how sure** it is — but it has no memory of the last frame.



One video frame. Every object the detector finds gets a **box** and a **label**.

Frame



Detector



Boxes

EACH DETECTION HAS

box

where it is

class

what it is

score

how sure (0 → 1)

A detector looks at **one frame at a time**. It does not know that a car here is the same car as in the last frame.

Multi-Object Tracking: Keep the Same ID

WHAT IT MEANS

A tracker connects the boxes from frame to frame and gives each object an **ID number**. Its job is to keep the **same number for the same object** the whole time.



The person beside the white car has **ID 1**.



The **same** person now has **ID 8** — an **identity switch**.

DETECTION ASKS

"What objects are here?"

TRACKING ASKS

"Which object is which, over time?"

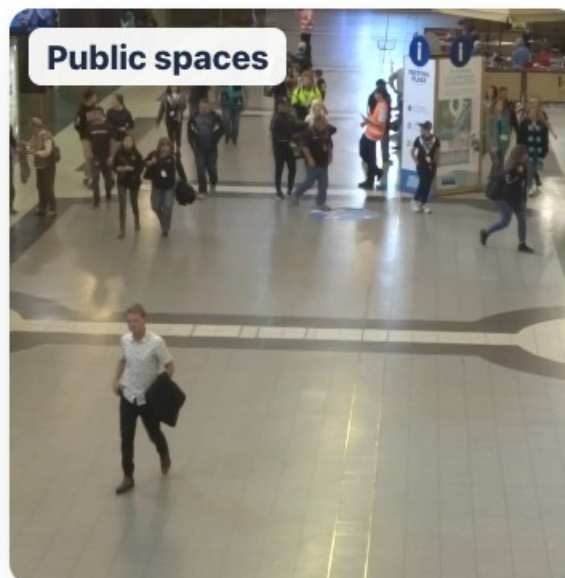
Where Multi-Object Tracking Is Used

THE MAIN IDEA

Tracking is used whenever we must **count, follow or watch** moving things: cars, crowds, people near roads — and objects seen from **drones**, which is our focus.



Count cars and measure traffic flow.



Follow crowds in stations and malls.



Watch people near roads and crossings.

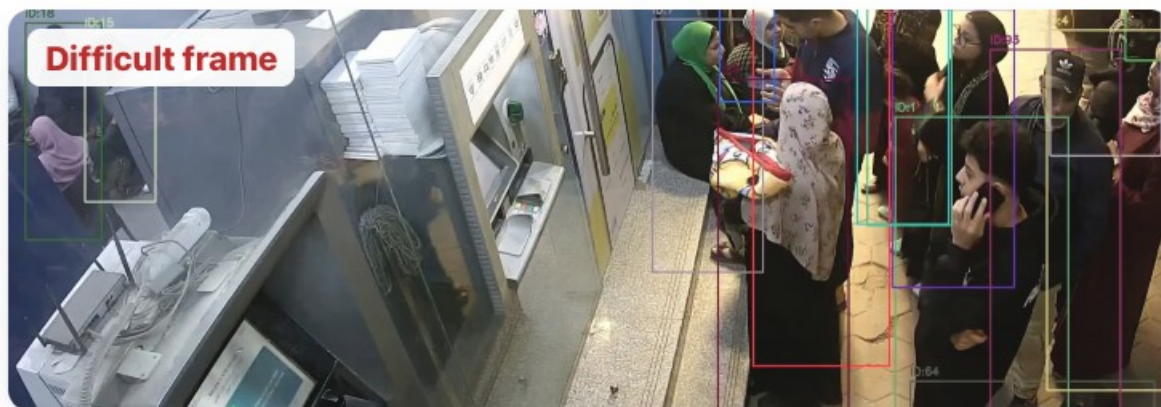


Our focus: drone video — tiny objects and a small onboard computer.

The Challenges of Real-Time Tracking

THE MAIN IDEA

The same camera sees **easy moments** and **hard moments**. Hidden, tiny, dark or blurred objects make a tracker **miss objects** and **swap IDs**.



Dense crowd, many hidden people, poor light. IDs are hard to keep.



Few objects, large, well separated, good light.

occlusion

small objects

crowds

camera motion

low light

motion blur

look-alike objects

Both frames can come from the **same video**, minutes apart.

How We Evaluate Tracking: Six Simple Questions

WHAT IT MEANS

No single number tells the whole story. We ask **six simple questions**: are the boxes right, did we find everything, is tracking accurate, do IDs stay the same, is it balanced, and is it fast?

DETECTION METRICS

1 · Are the boxes correct?

IoU · Intersection over Union

TP · true positive

FP · false positive

FN · false negative

2 · Are detections reliable and complete?

Precision

Recall

TRACKING METRICS

3 · Is tracking accurate overall?

MOTA · Multiple Object Tracking Accuracy

4 · Does each object keep its ID number?

IDF1 · Identity F1 score

IDS · identity switches

5 · Are finding and linking balanced?

HOTA · Higher Order Tracking Accuracy

SPEED METRIC

6 · Is it fast enough for live video?

FPS · frames per second

↑ **higher is better**: MOTA (Multiple Object Tracking Accuracy), HOTA (Higher Order Tracking Accuracy), IDF1 (Identity F1 score), FPS (frames per second)

↓ **lower is better**: IDS (identity switches), FP (false positive), FN (false negative)

IoU vs NMS — What Is the Difference?

WHAT IT MEANS

IoU is a **number** that says how much two boxes overlap. **NMS** is a **cleaning step** that deletes duplicate boxes — and it uses IoU to decide which ones.

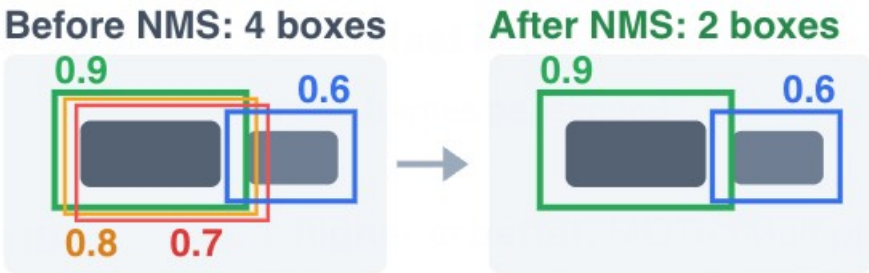
IOU · INTERSECTION OVER UNION

A **number from 0 to 1** that says how much two boxes overlap:
IoU = shared area ÷ total area of both boxes

IOU	WHAT YOU SEE	MEANING
0		the boxes do not touch — the box is wrong
0.5		half of the area is shared — the usual line for a correct box
1		the boxes lie exactly on top of each other — perfect
		Green = real object, red = detector box. In this picture: 0.40 = poor , 0.73 = good , 0.93 = excellent — the better the boxes match, the closer IoU gets to 1.

NMS · NON-MAXIMUM SUPPRESSION

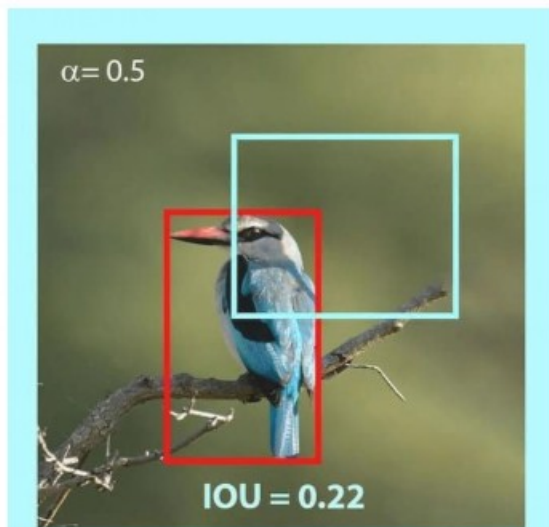
NMS keeps the **best box** and deletes its **copies** — it uses **IoU** to find them.



BOX	SCORE	IOU WITH 0.9 BOX	DECISION
A · green	0.9	—	keep · best score
A · orange	0.8	0.80	delete · copy
A · red	0.7	0.65	delete · copy
B · blue	0.6	0.05	keep · other car

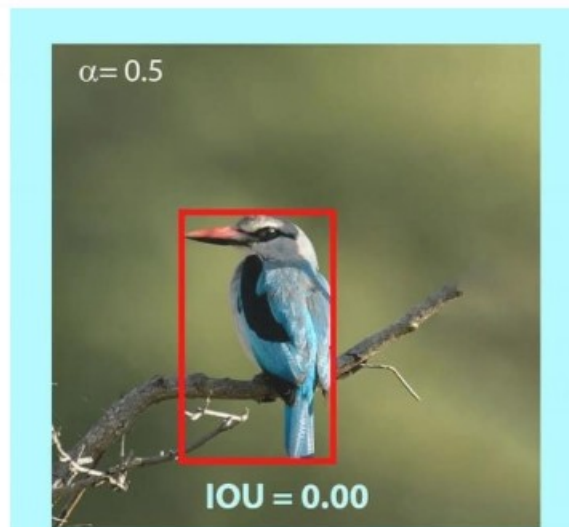
Rule: IoU with the kept box **above 0.45** = a **copy** → delete. **Below 0.45** = a **different object** → keep. **Result:** 2 cars → 2 boxes.

TP, FP, FN and TN: Detection Outcomes



False Positive

FALSE POSITIVE: a detection in the wrong location (IoU 0.22).



False Negative

FALSE NEGATIVE: the object is present but was not reported.

TP — True Positive

A real object was detected correctly.

FP — False Positive

The detector reported an object that was not really there.

FN — False Negative

A real object was present, but the detector missed it.

NOTE: There is a real bird in the image. **Box it correctly** and it is a **TP**. **Box empty background** and it is a **FP**. **Miss the bird** and it is a **FN**.

TN — True Negative: no object was present, and the detector correctly reported nothing.

Precision and Recall

PRECISION · CAN WE TRUST THE BOXES?

Of all the boxes the detector drew, **how many are correct?**

High precision = very few false alarms.

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

correct boxes ÷ all the boxes we drew

RECALL · DID WE FIND EVERYTHING?

Of all the real objects, **how many did the detector find?**

High recall = very few missed objects.

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

correct boxes ÷ all the real objects

EXAMPLE

A picture has **5 people**. The detector draws **5 boxes**: 4 are on people and 1 is on a shadow, and 1 person is missed.

TP (true positive) = 4 · FP (false positive) = 1 · FN (false negative) = 1 → Precision = $4 \div 5 = \mathbf{0.80}$ · Recall = $4 \div 5 = \mathbf{0.80}$

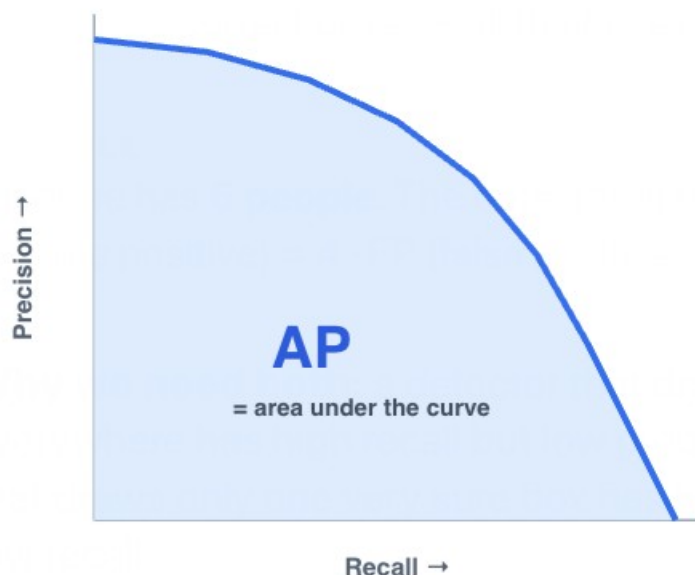
Why we need both: a detector that draws a box everywhere has high recall but low precision. A detector that draws only one very sure box has high precision but low recall.

Lower confidence threshold → the detector keeps more boxes → recall goes **up** (fewer misses), but precision often goes **down** (more false alarms).

Average Precision (AP) and Mean AP (mAP)

WHAT IT MEANS

AP shows how precise the detector stays while it finds more and more objects. AP is one score for **one class**, for example "car". **mAP** (mean Average Precision) is the **average AP over all classes**.



HOW TO READ THE CURVE

Recall goes to the right, precision goes up. A good detector keeps precision high even when recall is high, so the **area under the curve is big**. That area is AP, from 0 to 1.

EXAMPLE

Car AP = 0.90 and person AP = 0.80 $\rightarrow$ mAP = $(0.90 + 0.80) \div 2 = \mathbf{0.85}$

AP and mAP judge the **detector only**. They say nothing about keeping IDs over time.

MOTA — Multiple Object Tracking Accuracy

WHAT IT MEANS

MOTA says **how accurate the tracking is overall**. It starts from 100% and **subtracts every mistake**: missed objects, false boxes and ID switches. Higher is better.

$$\text{MOTA} = 1 - (\text{FN} + \text{FP} + \text{IDS}) / \text{GT}$$

all mistakes ÷ all real objects · FN = false negative (missed) · FP = false positive (false box) · IDS = identity switches · GT = ground truth (real objects)



Four people tracked with IDs 2, 3, 4 and 5.

EXAMPLE · MADE-UP NUMBERS

4 real people (GT = 4). The tracker misses 1 (FN = 1), draws 1 false box (FP = 1) and swaps 1 ID (IDS = 1).

$$\text{MOTA} = 1 - (1 + 1 + 1) \div 4 = 0.25$$

Watch out: MOTA adds all mistakes together. In crowded scenes misses are far more than ID switches, so MOTA can look good while IDs still switch.

Identity Switches (IDS)

WHAT IT MEANS

An identity switch happens when the tracker gives the same object a new ID number. IDS counts how many times this happens. **Lower is better — 0 is perfect.**

EXAMPLE



Before being hidden: **ID 4**.



After: the same person is now **ID 8** → one switch.

Most common cause: the object is hidden for a moment (behind a car or a tree). When it comes back, the tracker thinks it is a new object.

IDF1 — Identity F1 Score

WHAT IT MEANS

IDF1 says **how long each object keeps its correct ID**. 100% means every object kept the right ID the whole time. **Higher is better**.

EXAMPLE · ONE CAR APPEARS IN 10 FRAMES



Frames 1–8: correct ID kept

Frames 9–10: wrong ID

time with the correct ID / total time

$$8/10 = 80\%$$

Why we need it

- MOTA (Multiple Object Tracking Accuracy) can look good while IDs switch.
- IDF1 checks the identity directly.

HOTA and Processing Speed

HOTA · HIGHER ORDER TRACKING ACCURACY

One **balanced score** for tracking: did we **find** the objects, and did we **keep their IDs**? Higher is better.

$$\text{HOTA} = \sqrt{(\text{DetA} \times \text{AssA})}$$

DetA (Detection Accuracy) = did we find it? · **AssA** (Association Accuracy) = did we keep its ID?

Example · made-up numbers: DetA 0.64 and AssA 0.36 → $\text{HOTA} = \sqrt{(0.64 \times 0.36)} = \mathbf{0.48}$. If either part is bad, HOTA falls.

FPS · FRAMES PER SECOND

How many **video frames** the system handles in **one second**. Higher is faster.



FPS $\geq$ 25

our real-time rule

Example: the system handles 30 frames in 1 second → FPS = 30. At 25 FPS, each frame must take less than 40 ms.



SECTION II

Related Work: Detectors and Trackers

What we learned from our IEEE ICMISI 2026 survey: detector and tracker families, benchmarks, datasets, and the speed-vs-accuracy trade-off that showed us the gap.

Object Detectors Through the Ages

THE MAIN IDEA

Object detection moved from **rules made by people** to **deep learning** — and from **slow and careful** to **fast and accurate**.

2001 – 2012 ·
TRADITIONAL

Features **made by people**
+ a simple classifier.

Viola–Jones 2001

HOG + SVM 2005

DPM 2008

2014 – 2015 · TWO-STAGE

Deep learning: first find
possible regions, then
check each one.

R-CNN 2014

Fast R-CNN 2015

Faster R-CNN 2015

2016 → TODAY · ONE-
STAGE · OUR FAMILY

Deep learning: one look
gives boxes and classes
together.

YOLO 2016

SSD 2016

YOLOv3 2018

YOLOv8 2023

YOLO12 2025

2020 → TODAY ·
TRANSFORMERS

Deep learning with
attention: looks at the
whole image together.

DETR 2020

RT-DETR

RF-DETR 2025

Traditional

Hand-made features · runs on a CPU (central processing unit) ·
weak with small, crowded or unusual objects.

Deep learning

Features **learned** from many labelled images · needs a GPU
(graphics processing unit) · **much stronger** in hard scenes.

Three Families of Object Detectors

THE MAIN IDEA

Object detectors come in **three families**: one-stage (one fast look), two-stage (guess first, then check) and transformers (compare the whole image at once). We use **one-stage** because it is fast.

1 · ONE-STAGE · OUR FAMILY

Finds boxes and classes in **one direct step**.

YOLO SSD (Single Shot Detector)

RetinaNet

Simple example: YOLO (You Only Look Once) looks at the image once and says right away: car here, person there.

2 · TWO-STAGE

First guesses where objects might be, then checks each one.

Faster R-CNN Mask R-CNN

Simple example: Faster R-CNN (Region-based Convolutional Neural Network) says "maybe an object here", then fixes the box and decides the class.

3 · TRANSFORMER

Uses attention to look at the **whole image** together.

DETR Deformable DETR

RT-DETR (Real-Time DEtection TRansformer)

Simple example: DETR (DEtection TRansformer) compares all image parts together, then predicts objects with fewer repeated guesses.

Why we chose family 1

real-time speed small compute confidence can be tuned

works well with ByteTrack

Takeaway: one-stage is fastest, two-stage is more careful, transformers see the whole image.

Deep Learning Detectors: From Careful to Fast

THE MAIN IDEA

Deep learning **learns the features itself**. Two-stage models were accurate but slow; one-stage and new transformer models are **fast enough for live video**.

YEAR	DETECTOR	FAMILY	HOW IT WORKS	MAP · TIME	STRONG / WEAK
2014	R-CNN	two-stage	regions → a CNN on each	—	+ big accuracy jump · – very slow
2015	Faster R-CNN	two-stage	one network finds and checks	37.0 · 172 ms	+ accurate · – too slow for video
2016	YOLO	one-stage	one look, a grid of boxes	—	+ very fast · – misses small objects
2018	YOLOv3	one-stage	boxes at three sizes	31.0 · 28.6 ms	+ real-time · – lower accuracy
2020	DETR R101	transformer	attention on the whole image	42.9 · 145 ms	+ simple design · – very slow
2023	YOLOv8n	one-stage	small YOLO, no anchor boxes	37.3 · 3.2 ms	+ tiny, fast · – less accurate
2025	YOLO12m	one-stage	YOLO with attention	52.5 · 4.86 ms	+ accurate, real-time · – bigger
2025	RF-DETR-S	transformer	a real-time DETR	52.9 · 3.5 ms	+ best trade-off · – bigger

Best in our table: **RF-DETR-S** (52.9 mAP at 3.5 ms). Our choice: **YOLOv8n** — tiny, fast and easy to control.

- mAP and time: published values in our IEEE ICMISI 2026 survey (COCO val2017 · T4 · TensorRT FP16 · 640 × 640) · — = not in our benchmark

Top Detectors: Benchmark Comparison

THE MAIN IDEA

This table compares famous detectors **tested in exactly the same way**. The key point: **YOLOv8n is about as accurate as Faster R-CNN** (Region-based Convolutional Neural Network), **but about 54x faster** and much smaller.

YEAR	MODEL	TYPE	MAP	SPEED	SIZE	GOOD FOR
2015	Faster R-CNN	Two-stage	37.0	172 ms	41.8M	accurate, too slow for live video
2018	YOLOv3	One-stage	31.0	28.6 ms	61.9M	the first truly real-time one
2023	YOLOv8n	One-stage	37.3	3.2 ms	3.2M	best balance — our choice
2023	YOLOv8s	One-stage	44.9	12.5 ms	11.2M	more accurate, 4x slower
2025	YOLO12m	One-stage	52.5	4.86 ms	20.2M	real-time accuracy leader
2020	DETR (DEtection TRansformer) R101	Transformer	42.9	145 ms	60.6M	elegant, but very slow
2025	RF-DETR-S	Transformer	52.9	3.5 ms	32.1M	best published trade-off

All numbers on COCO (Common Objects in Context) val2017, T4 GPU (graphics processing unit), TensorRT FP16 (16-bit floating point), 640 × 640, batch 1 — so they can be compared directly.

Compare row 1 with our row: Faster R-CNN and YOLOv8n are almost equally accurate (37.0 vs 37.3 mAP), but YOLOv8n is **54x faster** (3.2 vs 172 ms) and **13x smaller**.

Why We Chose YOLOv8n

THE MAIN IDEA

We picked **YOLOv8n** because it is **small, fast and accurate enough** — and we keep the **same detector in every test**, so any gain comes from our method, not from the detector.

MAP (MEAN AVERAGE PRECISION)
ON COCO (COMMON OBJECTS IN
CONTEXT)

37.3

PER FRAME ON A T4

3.2 ms

PARAMETERS

3.2M

DETECTOR HEADROOM

~300 FPS

Why YOLOv8n fits

- Fast enough for real time on one GPU (graphics processing unit).
- Small and light for real use.
- Leaves computing room for ByteTrack and bigger inputs.
- Accurate enough to test our idea fairly.

Why not a bigger YOLO?

- Our contribution is the adaptive system, not a bigger detector.
- The same detector keeps every comparison fair.
- A bigger model costs more memory and time.
- Our method can be tested with other YOLO (You Only Look Once) models later.

Same detector in every run

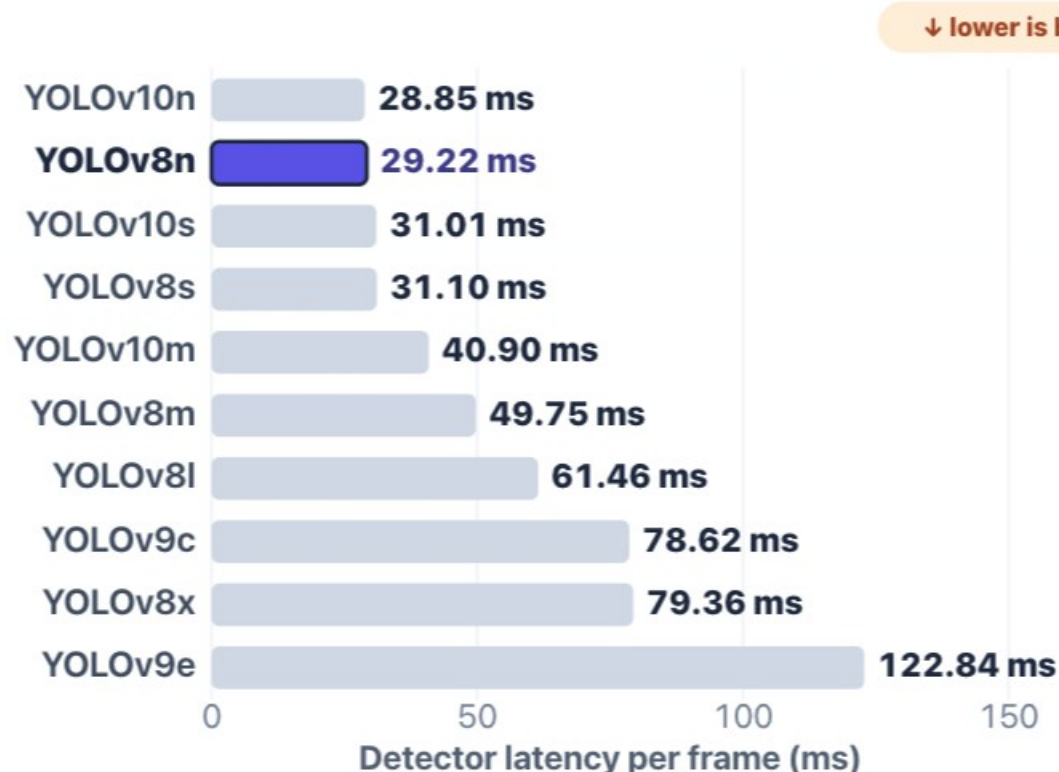
Fair comparison

Gains come from our method, not from a bigger model

Our Own Speed Test of YOLO Models

THE MAIN IDEA

We also timed YOLO (You Only Look Once) models ourselves: **smaller models run faster**. This early test agrees with the published table, but its raw log was not kept, so it is support only.



The small “n” models are the fastest. For YOLOv8, the largest model needs about **2.7× more time** per frame than the smallest.

This is the detector alone. The full system also runs scene analysis and tracking, so it needs this spare time.

Three Families of Trackers

THE MAIN IDEA

Trackers also come in three families. We use **tracking-by-detection**: first find the boxes, then **link them across frames** — simple, fast and easy to control.

1 · TRACKING-BY-DETECTION · OUR FAMILY

A detector finds boxes. A tracker links them across frames.

SORT (Simple Online and Realtime Tracking)

DeepSORT

ByteTrack

2 · JOINT DETECTION + REID

One network finds objects **and** learns how they look (ReID = re-identification).

FairMOT

JDE (Joint Detection and Embedding)

CenterTrack

3 · TRANSFORMER TRACKERS

Attention looks at the whole frame and links objects together.

MOTR

TrackFormer

OUR CHOSEN PIPELINE

Video frame



Detector · boxes



Tracker · links IDs



Tracks

Why we chose family 1

fast for real-time video

detector and tracker improve separately

easy to explain and debug

Takeaway: detect first, then keep the same ID over time. It is fast, modular and clear for our work.

How Trackers Evolved: SORT to ByteTrack

THE MAIN IDEA

Trackers improved step by step: SORT (Simple Online and Realtime Tracking) used only **position**, DeepSORT added **appearance**, and ByteTrack also keeps **weak boxes**, so half-hidden objects keep their ID.

2016

SORT

Uses box position only. Very fast, but IDs can switch.

2017

DeepSORT

Adds appearance features. Fewer switches, but slower.

2022

ByteTrack

Keeps weak boxes too. Fast and strong for real time.

2022

BoT-SORT (Bag of Tricks SORT)

Adds camera-motion help. Strong, but more complex.

WHAT EVERY TRACKER DOES

1 · Create track



2 · Update ID



3 · Keep alive



4 · Delete

A track starts when an object appears, then keeps the same ID while it moves.

Why ByteTrack fits our work

real-time speed

simple design

uses low-score boxes

works with adaptive detector settings

Takeaway: SORT is fastest but simple. DeepSORT adds identity. ByteTrack keeps the speed and recovers more objects.

ByteTrack: The Baseline of This Work

THE MAIN IDEA

ByteTrack's job is to keep the **same ID** for an object — even when the detector is **unsure** (low score) or **misses it** for a few frames.

Strong box · score ≥ 0.50
matched to the tracks **first**

Weak box · score 0.10 – 0.50
gets a **second chance**

No box
keep the track and **wait** up to 30 frames

EXAMPLE · ONE CAR WITH ID 7 DRIVES BEHIND A WALL

Frame 100



0.85 · **strong**
matched first

ID 7

Frame 101



0.72 · **strong**
matched first

ID 7

Frame 102



0.30 · **weak** — half hidden
second chance: fits the
dashed predicted box (overlap
0.75) → same car

ID 7

Frames 103–105



no box — fully hidden
track buffer: keep ID 7 and
wait

ID 7 · waiting

Frame 106



box again
near the predicted place → **the
old track**

ID 7

ByteTrack: ID 7 → 7 → 7 → 7 → 7 · **the same ID the whole time** ✓

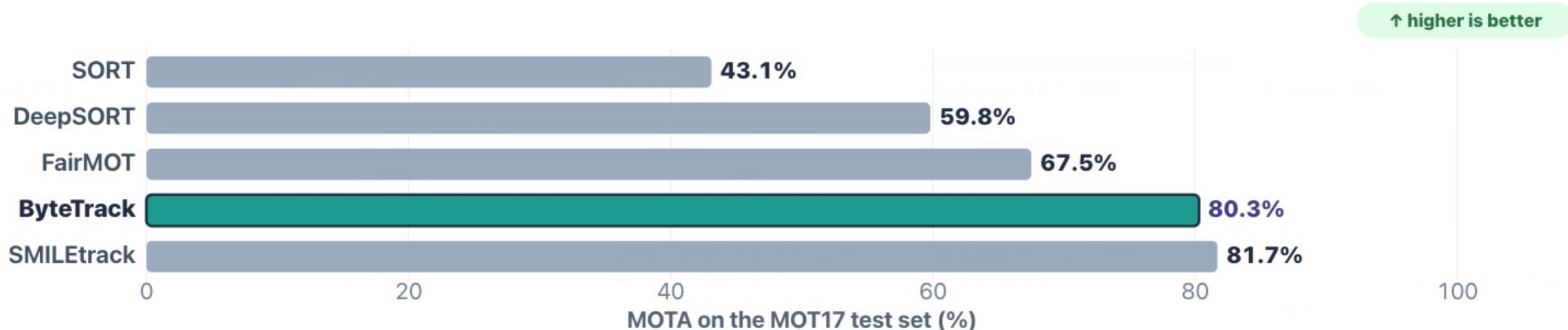
A simple tracker (drops weak boxes, no waiting): ID 7 → 7 → lost → lost →
new ID 8 × = an **ID switch**

- Worked example · illustration settings 0.50 / 0.10 / 30 and the simple-tracker line are illustrations · our tuned ByteTrack uses 0.18 / 0.04 / 45

Published Tracking Accuracy (MOTA) on MOT17

THE MAIN IDEA

On the MOT17 street benchmark, published trackers reach about **80% MOTA** (Multiple Object Tracking Accuracy). But people there are large and close to the camera — **drone video is much harder**, so our numbers will be lower.



Note: MOT17 is an easier benchmark. Our numbers later are much lower because drone video is much harder, so comparing them directly is not fair.

MOTA alone hides ID mistakes. A tracker can score 80% and still rename the same car three times. So next we look at ID switches.

QUICK QUESTION



Which Tracker Switches IDs the Least?

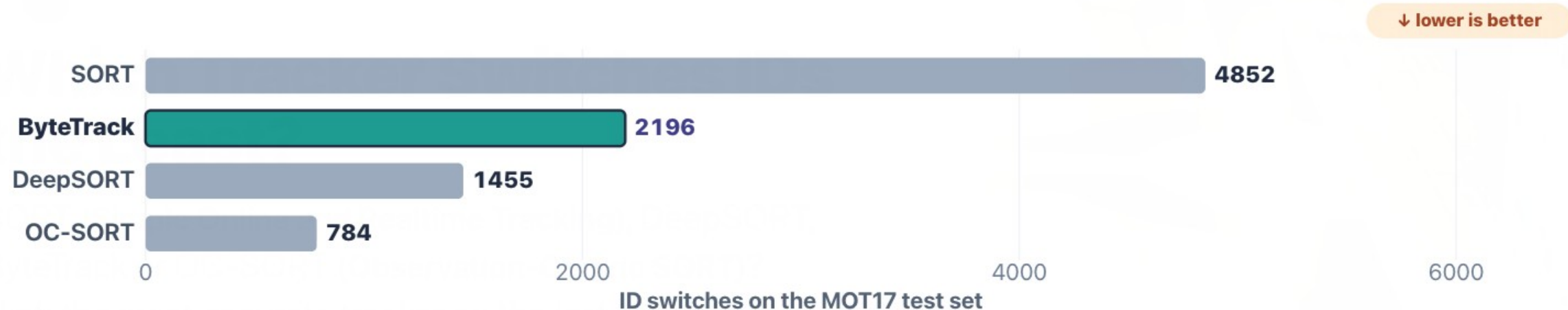
SORT (Simple Online and Realtime Tracking), DeepSORT, ByteTrack or OC-SORT (Observation-Centric SORT)?

Hint: the most accurate tracker on the last slide is not the winner. The answer is on the next slide.

The Same Trackers, Judged on ID Switches

THE MAIN IDEA

Judged on **ID switches**, the ranking changes: the most accurate tracker is **not** the one that keeps IDs best. There is a **trade-off between tracking accuracy (MOTA = Multiple Object Tracking Accuracy) and ID switches** — **one number is never enough**.



Trade-off: the order flipped. ByteTrack has the **highest MOTA** of these trackers (80.3) but **not the fewest ID switches** (2,196 vs 784 for OC-SORT). Higher tracking accuracy does **not** always mean more stable IDs.

ByteTrack is not the lowest-IDS tracker. We chose it for balance, and reducing its ID switches became one goal of this work.

What the Survey Taught Us

THE MAIN IDEA

Detectors became **much faster and more accurate**, but **no model is best for every job**. For real-time drone video we chose a **light, fast pair**: YOLOv8n + ByteTrack.

SPEED · TIME PER IMAGE

~49x faster

Faster R-CNN (Region-based Convolutional Neural Network) (2015)
 $172 \text{ ms} \rightarrow \text{RF-DETR-S (2025)} \text{ } 3.5 \text{ ms} \cdot 172 \div 3.5 \approx 49$

ACCURACY · COCO (COMMON OBJECTS IN CONTEXT) MAP (MEAN AVERAGE PRECISION)

+43% mAP

Faster R-CNN (2015) $37.0 \rightarrow \text{RF-DETR-S (2025)} \text{ } 52.9 \cdot (52.9 - 37.0) \div 37.0 \approx 43\%$

TRACKING ACCURACY · MOT17

81.7% MOTA

the **highest MOTA** (Multiple Object Tracking Accuracy) in our tracker table: **SMILEtrack**

FASTEST DETECTOR

3.5 ms

per image: **RF-DETR-S**, the fastest in the detector table

Survey conclusion: no single model is best everywhere. The right choice balances speed, compute and accuracy.

Our chosen baseline

After comparing detector and tracker pairs, **YOLOv8n + ByteTrack** gave the best trade-off for a real-time, modular system.

What this work tests

Other models may score higher in some cases. Our goal is to test whether **scene-based adaptive settings** improve tracking while staying real-time and cheap.

SECTION III

The Problem and Our Starting Point

Drone scenes change from easy to hard — but a normal pipeline keeps one fixed detector setting. First, a baseline to measure against.



One Drone Video, Very Different Frames

THE MAIN IDEA

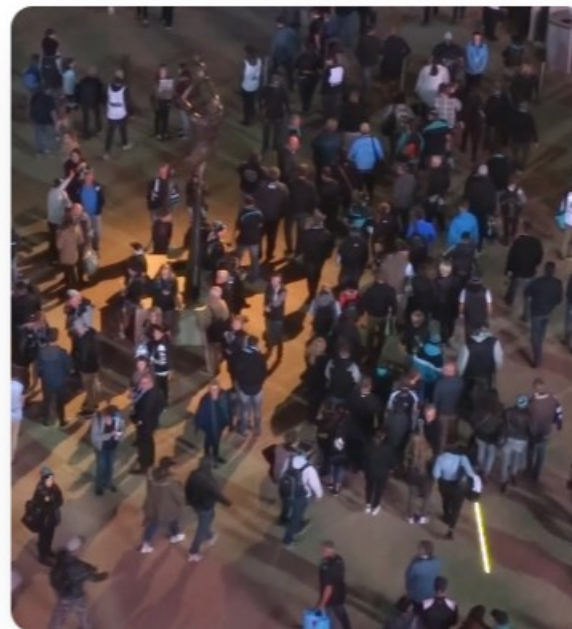
In the **same drone video**, some frames are **easy** and some are **hard**.

Easy frame



- **few** objects
- **big** objects
- **good** light
- **sharp** picture

Hard frame



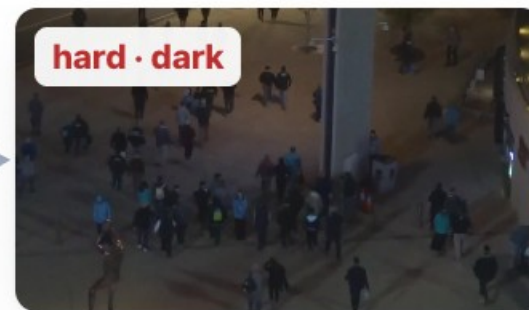
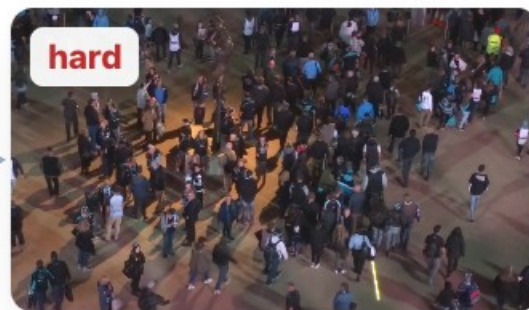
- **many** objects
- **tiny** objects
- **busy** background
- **dark** or **blurry**

As the drone flies, the scene can change from **easy** to **hard** and back again.

The Problem: The Detector Never Changes Its Settings

THE MAIN IDEA

A normal detector uses the **same settings for every frame** — easy or hard.



same settings for all frames →

same confidence threshold

same NMS (Non-Maximum Suppression) IoU (Intersection over Union)

same input size

On a hard frame

Settings that are too strict **miss weak and tiny objects**.

On an easy frame

Extra effort, like a bigger input image, only **costs time**.

One fixed setting **cannot be right for every frame**.

The Main Question

OUR QUESTION

If the scene keeps changing, **why do the detector settings stay the same?**

Confidence threshold

How **sure** the detector must be to keep a box.

NMS (Non-Maximum Suppression) **IoU** (Intersection over Union)

How much two boxes may **overlap** before one is removed as a copy.

Input size

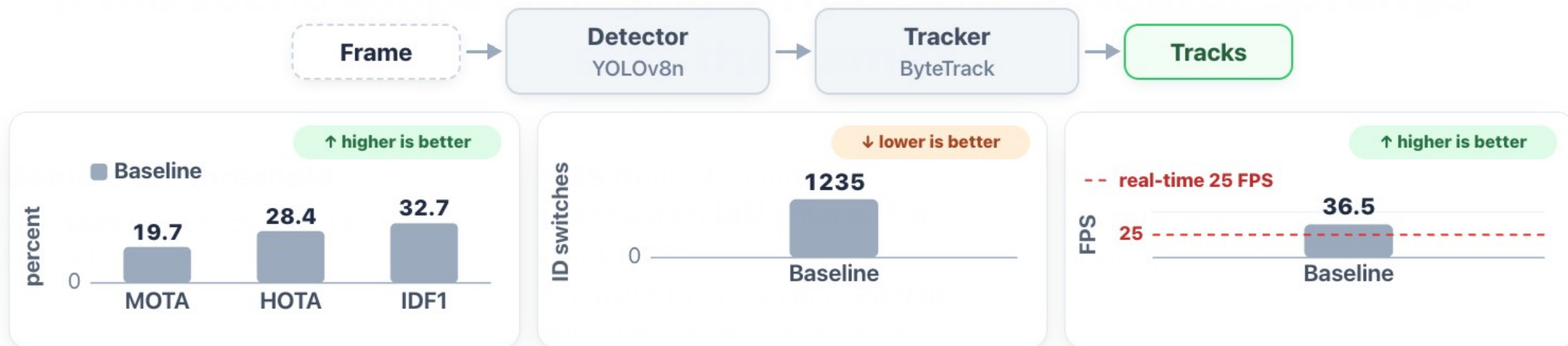
How **big** the image is when the detector looks at it.

Our idea: let these settings **follow the scene** — automatically, for every frame, **without retraining** the detector.

Before Adding Anything: A Baseline

THE MAIN IDEA

First we need a **simple, fast starting system**. Every later gain is measured **against it**.



Why this pipeline?

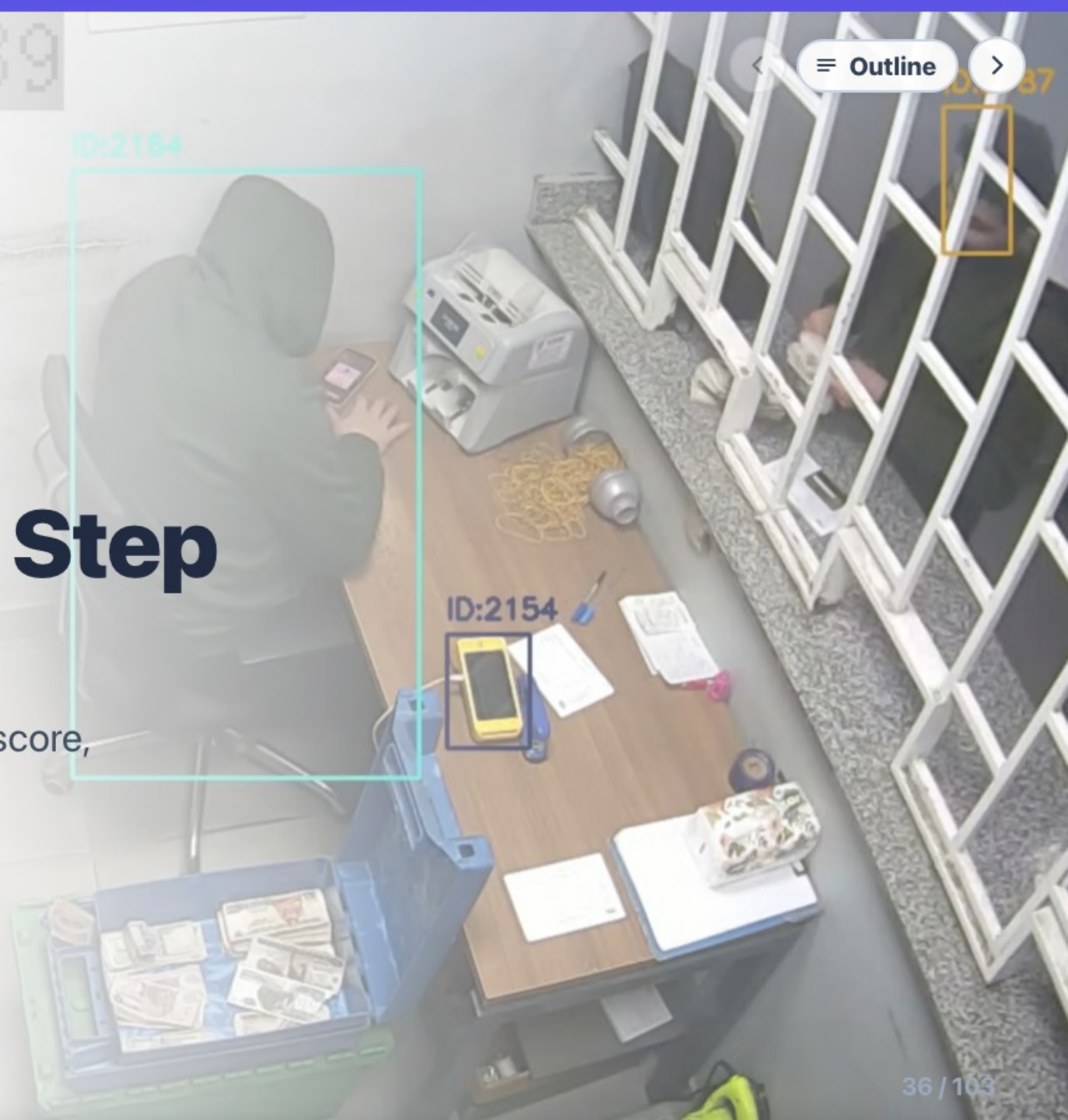
- **fast** — real-time on drone video
- **modular** — detector and tracker are separate parts
- anything we add around them can be **measured**

This is **not** ByteTrack's global score. It means: YOLOv8n + ByteTrack, **with our settings and our test protocol**, gives 19.729 MOTA (Multiple Object Tracking Accuracy).

IV SECTION IV

AC-MOT: Our Idea, Step by Step

Measure how hard the frame is, turn it into one score, then set the detector to match.



The Idea: Measure the Scene, Then Set the Detector

THE MAIN IDEA

AC-MOT (Adaptive Control for Multi-Object Tracking) adds **three small steps before the detector**. For every frame it **measures** how hard the scene is, turns that into **one score**, and **sets the detector** for it — like a driver who goes **slowly in fog** and **faster on an empty road**.

From a Frame to SCI — All Steps Together

THE MAIN IDEA

Seven small steps turn a video frame into **one stable difficulty number**: shrink it, measure five cues, mix them, keep the result between 0 and 1, and average the last readings.

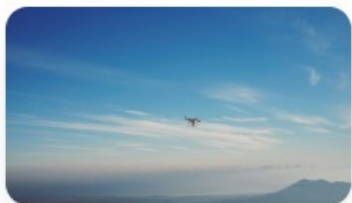
Read it left to right: prepare the frame → measure difficulty → mix and clip → smooth 7 readings → the final SCI (Scene Complexity Index) sets the detector for the next frame.

Step 1: First, See the Cues in Real Pictures

THE MAIN IDEA

Before any formula, just **look**: a busy scene has **many edges**, a night picture is **darker**, and a blurred picture has **no sharp details**.

EDGES · BUSY OR CALM?



Calm sky: edge density **0.021**



Busy parking lot: edge density **0.295**

NIGHT · IS IT DARK?

Day



mean gray **110.6** → night flag **0**

Night



mean gray **40.5** → night flag **1**

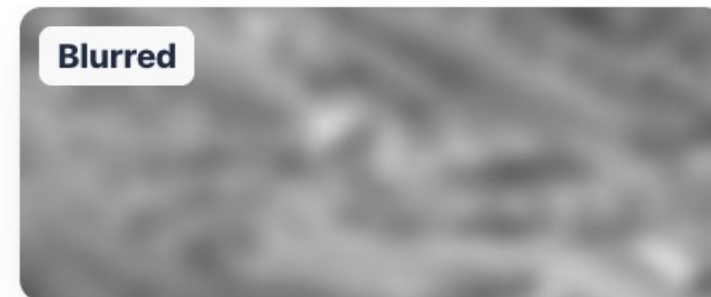
BLUR · SHARP OR SMEARED?

Sharp



Laplacian variance **4,074** → blur flag **0**

Blurred



Laplacian variance **4** → blur flag **1**

• Computed on these example images at 25% size (tools/gen_figures.py) · edge maps use a simple edge filter for illustration

Step 1: Two Cues From the Detector's Boxes

THE MAIN IDEA

Two cues use the **boxes found in the previous frame**: how many objects are there, and how **small** are they?

CROWD · HOW MANY OBJECTS?

count the boxes → ÷ 30 → stop at 1

BOXES	CROWD
3	$3 \div 30 = \mathbf{0.10}$
15	$15 \div 30 = \mathbf{0.50}$
30	$30 \div 30 = \mathbf{1.00}$
45	more than 30 → still 1.00
30 or more objects already counts as fully crowded .	

TINY · ARE THE OBJECTS VERY SMALL?

look at every box → count boxes under 32×32 px →

÷ all boxes

BOXES	TINY
10 boxes, 2 tiny	$2 \div 10 = \mathbf{0.2}$
10 boxes, 7 tiny	$7 \div 10 = \mathbf{0.7}$

32×32 pixels is how the COCO (Common Objects in Context) dataset defines a **small object**.

Both cues give a number from **0** (easy) to **1** (hard).

Step 1: Three Cues From the Picture Itself

THE MAIN IDEA

Three cues look at a **small gray copy of the frame** and ask: is it **busy, dark or blurry**?

EDGES · IS IT BUSY?

Canny is a standard method that **draws the edges** of a picture: the places where brightness **changes sharply**, like object borders and road lines.

Edge density = edge pixels ÷ all pixels.

Edges = edge density ÷ 0.14, at most 1
→ **14%** edge pixels already counts as **fully busy**.

NIGHT · IS IT DARK?

Every pixel has a **brightness** from **0** (black) to **255** (white).

We take the **average brightness** of the whole picture.

Night = **1** if the average is below **80**, otherwise 0.

BLUR · IS IT BLURRY?

The **Laplacian** is a filter that gives a **big answer at sharp details** and a small answer on smooth areas.

Its **variance** (spread) is **big for a sharp picture** and **small for a blurry one**.

Blur = **1** if the variance is below **180**, otherwise 0.

30, 32 × 32, 0.14, 80 and 180 are fixed design constants (only 32 × 32 comes from COCO). **Optuna did not change them.**

Canny, IEEE TPAMI 1986 · variance of the Laplacian: Pech-Pacheco et al., ICPR 2000 · small object: Lin et al., "Microsoft COCO," ECCV 2014

Step 2: One Difficulty Score — the SCI



SCENE COMPLEXITY INDEX (SCI)

One number from 0 to 1 that says how hard the current frame is for the detector.

How to read it

- Near 0.0 → **CLEAR**: open road, few large objects, good light
- Around 0.5 → **MEDIUM**: some crowding or clutter
- Near 1.0 → **CROWDED**: dense, tiny, dark or blurred

Computed every 10 frames on a small 25% grayscale copy, using OpenCV and NumPy on the CPU (central processing unit) — under 0.2 ms, with no GPU (graphics processing unit) time taken from the detector.

Note: the dial reads 0.63 for a crowded frame with many small objects and poor light. SCI does not detect anything itself. It only says how hard the next frame is, so the detector can be set up for it.

Step 2: How the Five Cues Become SCI

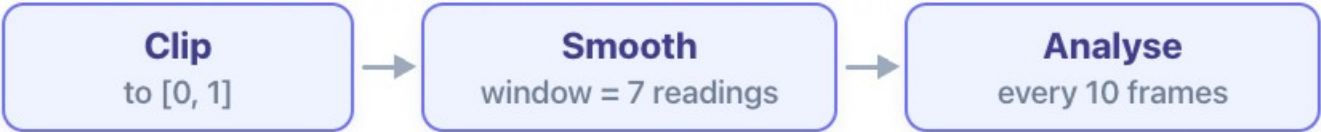
THE MAIN IDEA

The first AC-MOT (Adaptive Control for Multi-Object Tracking) design mixed the five cues with **hand-picked weights** into one number, the SCI (Scene Complexity Index), and then kept that number **stable over time**.

$$SCI = 0.30 \cdot \text{Crowd} + 0.30 \cdot \text{Tiny} + 0.20 \cdot \text{Edge} + 0.10 \cdot \text{Night} + 0.05 \cdot \text{Blur}$$

CUE	INITIAL WEIGHT
Crowd	0.30
Tiny	0.30
Edge	0.20
Night	0.10
Blur	0.05

THEN, TO KEEP SCI STABLE



This was the first heuristic AC-MOT design. The weights were chosen by hand — they even add up to 0.95, not 1.

Step 2: Example — SCI = 0.63

THE MAIN IDEA

To get SCI (Scene Complexity Index), **multiply each cue by its weight and add them up**. Here a crowded, tiny and dark scene gives **0.63** — a hard frame.

CUE	VALUE	WHAT WAS SEEN
crowd	0.7	21 objects in the frame ($21 \div 30$)
tiny	0.8	8 of every 10 boxes are under 32 px
edges	0.4	busy road texture and markings
night	1	average brightness 60 — dark
blur	0	the frame is sharp

$$\begin{aligned}\text{SCI} &= 0.30 \times 0.7 + 0.30 \times 0.8 + 0.20 \times 0.4 + 0.10 \times 1 + \\ &\quad 0.05 \times 0 \\ &= 0.21 + 0.24 + 0.08 + 0.10 + 0 = \mathbf{0.63}\end{aligned}$$

SCI

0.63

a hard frame

Crowded, tiny and dark add up. Blur adds nothing because the frame is sharp.

Step 2: Keeping SCI Stable and Cheap

THE MAIN IDEA

Two tricks keep it **stable and cheap**: **average the last 7 SCI** (Scene Complexity Index) **readings**, so one odd frame changes nothing, and **analyse only every 10th frame**.

1 · The 7-reading rolling mean

Without it, SCI would jump between CROWDED and CLEAR from frame to frame, and the settings would swing back and forth. That hurts the tracker.

It costs nothing: a 7-value running average.

2 · Analyse only every 10 frames

Scenes change over seconds, not milliseconds. Checking every 10th frame is enough. The result is reused in between.

10

frames between analyses

25%

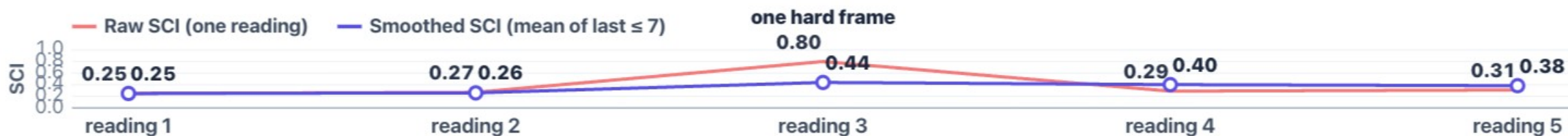
grayscale copy size

0.2 ms

per frame, CPU (central processing unit) only

0

GPU (graphics processing unit) time taken



Analysing a small gray copy only every 10 frames keeps the extra cost very low.

Step 3: The Three Detector Settings We Change

THE MAIN IDEA

Each setting has a job — and **both too high and too low cause a problem** for the objects.

Confidence threshold

How **sure** must the detector be to keep a box?

too high weak but **real** objects are **missed**

too low **false boxes** on shadows and noise

Example: a tiny car scores 0.4. Threshold 0.5 → the car is **missed**. Threshold 0.2 → the car is kept, but a shadow scoring 0.25 is kept too.

NMS (Non-Maximum Suppression) IoU (Intersection over Union)

When are two overlapping boxes **the same object**?

too high **copies stay** — one car counted twice

too low close **real** objects are **deleted**

Example: two boxes on one car overlap 0.6. Limit 0.45 → the copy is removed. Limit 0.7 → both stay. Two people standing close (overlap 0.5) with limit 0.45 → one **real person is deleted**.

Input size

How **big** is the image the detector sees?

too small tiny objects **vanish**

too big every frame is **slower**

Example: a person only a few pixels wide. Small input (640) → the person can **disappear**. Big input (832) → still visible, but the frame takes **more time**.

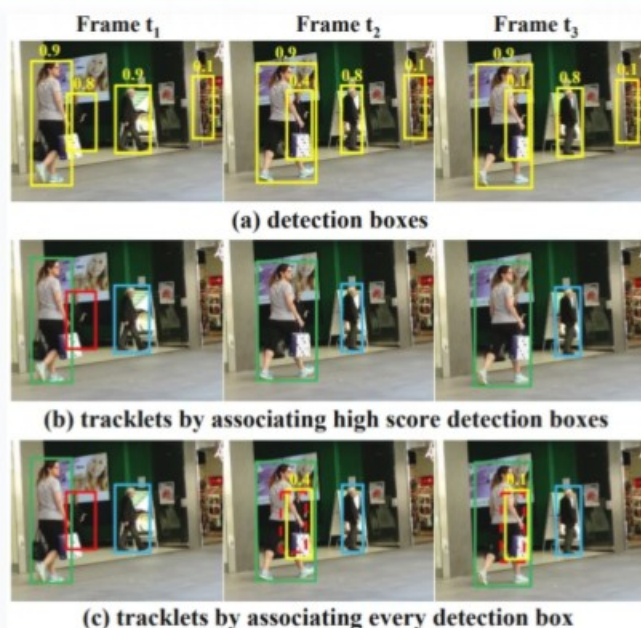
Hard frame → lower confidence · stronger copy removal · bigger input

Easy frame → stay strict, small and fast

Step 3: The Confidence Threshold in Detail

WHAT IT MEANS

The **confidence threshold** decides which boxes the detector keeps. **High** = clean but misses weak objects; **low** = finds more but lets noise in. The best line depends on the scene.



Row (a): the detector's own scores above each box — 0.9, 0.8, 0.4 and 0.1.

High threshold (e.g. 0.5)

Keeps only 0.9 and 0.8. Clean, but the weak real object (0.4) is lost.

precision ↑

recall ↓

Low threshold (e.g. 0.2)

Also keeps 0.4. The object is found, but more false boxes can enter.

recall ↑

false positives ↑

The best line depends on the scene — so AC-MOT (Adaptive Control for Multi-Object Tracking) moves it with SCI (Scene Complexity Index).

Step 3: From SCI to Detector Settings

THE MAIN IDEA

In the original design, a **harder frame** gets a **lower confidence** (keep weak but real boxes) and a **bigger input size** (more pixels for tiny objects). Easy frames stay fast.

SCI	LABEL	CONF	INPUT SIZE	WHAT IT MEANS
below 0.35	CLEAR	0.245	640	easy frame — stay strict and fast
0.35 to 0.60	MEDIUM	~0.228	736	some crowding — loosen a little
0.60 and up	CROWDED	0.202	832	hard frame — keep weak boxes, zoom in
1.00	CROWDED	0.190 (floor)	832	worst case — never below 0.190

Confidence goes DOWN

In a hard scene, a weak detection is usually a real small object, not noise. Lowering the line saves it.

Resolution goes UP

A 5–10 pixel object needs more pixels to be seen at all. Only hard frames pay that cost.

Note: both moves fix the same problem. Lower confidence keeps the weak but real box. A bigger input gives it more pixels. Easy frames get neither, so they stay fast.

Step 3: The Exact Rules

THE MAIN IDEA

The settings change **smoothly with SCI** (Scene Complexity Index): confidence and IoU (Intersection over Union) slide down a little as the scene gets harder, and the input size **jumps up in two steps**.



SETTING	RULE
confidence	$0.245 - 0.050 \times \text{SCI}$, kept between 0.19 and 0.28
NMS (Non-Maximum Suppression) IoU	$0.490 - 0.050 \times \text{SCI}$, kept between 0.40 and 0.52
input size	832 if $\text{SCI} > 0.60$ · 736 if $\text{SCI} > 0.35$ · otherwise 640

SMALL FIX	WHEN
confidence - 0.012	the scene is crowded, tiny or dark
NMS IoU - 0.012	the scene is blurred
input size = 832	more than half of the boxes are tiny
then	the limits are applied again

Step 3: What Changes and What Never Changes

THE MAIN IDEA

Only **three detector settings** change every frame. The tracker settings are **tuned once and then frozen**. Do not mix the detector's NMS (Non-Maximum Suppression) **IoU** (Intersection over Union) with the tracker's match threshold.

DETECTOR SIDE · MOVES EVERY FRAME

- **conf** — the keep-or-drop line for a box
- **iou** — the NMS overlap threshold
- **imgsz** — the size the frame is fed in at

Passed straight into `model.track(conf=, iou=, imgsz=)`.

TRACKER SIDE · TUNED ONCE, THEN FROZEN

track_high_thresh	0.18	track_low_thresh	0.04
new_track_thresh	0.20	track_buffer	45
match_thresh	0.86		

Written into the ByteTrack YAML. Never adaptive.

There are **two different IoU values**. The calibrator moves the **detector's NMS** threshold. The one that matches tracks to boxes is **match_thresh = 0.86**, and it never changes.

What NMS does

Lowering the NMS IoU makes suppression STRONGER — more overlapping boxes are deleted.

Why tighten it in hard scenes?

We already lowered confidence and raised the size, so more duplicates appear. Tightening removes them.

V SECTION V

Experimental Setup and the First Result

VisDrone2019-MOT, the scoring rules — and the first answer: did the hand-designed AC-MOT (Adaptive Control for Multi-Object Tracking) help?

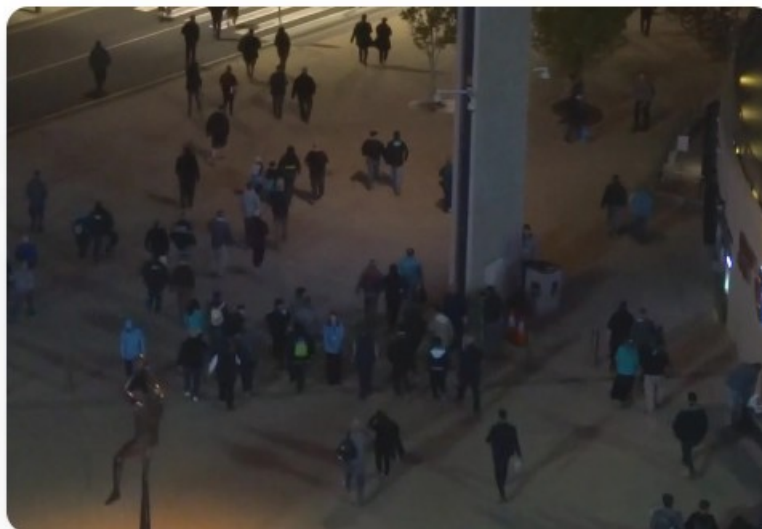
Choosing the Test Domain: Drone Surveillance

THE MAIN IDEA

To test the method we pick the **hardest domain: drone video**. Objects are tiny, and one flight goes from **very hard to very easy** scenes.



From high up, a person is a blob about ten pixels wide.



The same kind of scene at night: contrast falls away.



Minutes later, an open road with large, easy objects.

small objects

dense objects

camera motion

scale changes

occlusion

The VisDrone2019-MOT Benchmark

THE MAIN IDEA

VisDrone2019-MOT is **Full-HD drone video** with objects only **5–30 pixels wide**. We **tune on 7 validation videos** and **test on 17 test-dev videos**.

Whole benchmark	288 video clips · 262K frames (all VisDrone2019 tasks)
Resolution	1920 × 1080 (Full HD)
Altitude	5 to 120 m above the ground
Camera angle	straight down to tilted
Object size	5 to 30 pixels wide — extremely small
Conditions	daylight, low light, motion blur
Class balance	Natural benchmark distribution · not designed with equal class counts
Motion source	Both camera motion and object motion · observed motion can combine both
Tuning split	validation · 7 sequences
Final test split	test-dev · 17 sequences · 6,635 frames
Used in this work	7 + 17 = 24 sequences of the tracking (MOT = Multi-Object Tracking) part
Density	32.4 objects per frame (215,014 filtered boxes)



● Split sizes and box count from the final evaluation runs (TrackEval GT count) · whole-benchmark size from our survey table

Object Classes We Evaluate

THE MAIN IDEA

We score **five kinds of objects**: pedestrian, car, van, truck and bus. Tiny bikes and tricycles are left out, because even people cannot label them clearly.

ID	VISDRONE CLASS	USED?
1	Pedestrian	YES
2	People (group)	no
3	Bicycle	no
4	Car	YES
5	Van	YES
6	Truck	YES
7	Tricycle	no
8	Awning-tricycle	no
9	Bus	YES
10	Motor	no

Why exactly these five

- The reference papers on this dataset use the same classes, so our comparison with them is fair.
- They are the real targets of drone surveillance.
- They have enough labels for stable numbers.
- YOLOv8n finds them well from above.

Why the others are left out

- A bicycle at 100 m is a few unclear pixels, hard to tell from a motorbike.
- Even human labellers disagree on them.

Honest note: our COCO (Common Objects in Context) detector has no separate “van” class, so some vans may count as misses. This is the same for every system we compare.

Ground-Truth Filtering Rules

THE MAIN IDEA

Before scoring, we **remove labels that are too hidden or too cut off**. Otherwise a tracker would be punished for missing things a person can hardly see.

OCCLUSION	MEANING	KEPT?	TRUNCATION	MEANING	KEPT?
0	fully visible	KEPT	0	fully inside the frame	KEPT
1	slightly hidden	KEPT	1	a small part outside	KEPT
2	heavily hidden	removed	2	a large part outside	removed
3	almost fully hidden	removed	3	mostly outside	removed

Why remove the hard ones

If a box is not clear enough for a person to label well, it is unfair to count it as a tracking error. That would measure label doubt, not tracker quality.

The annotation score field

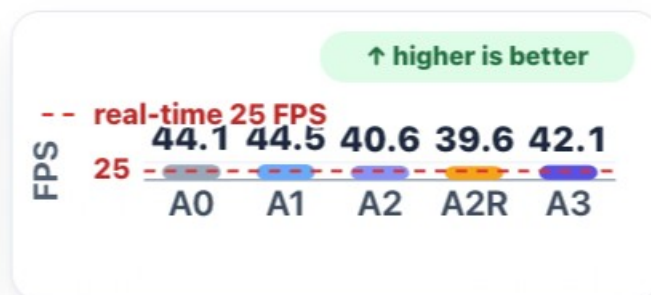
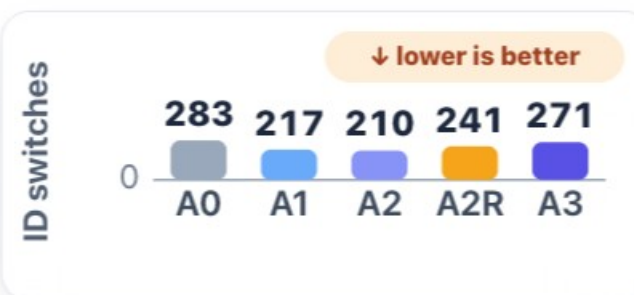
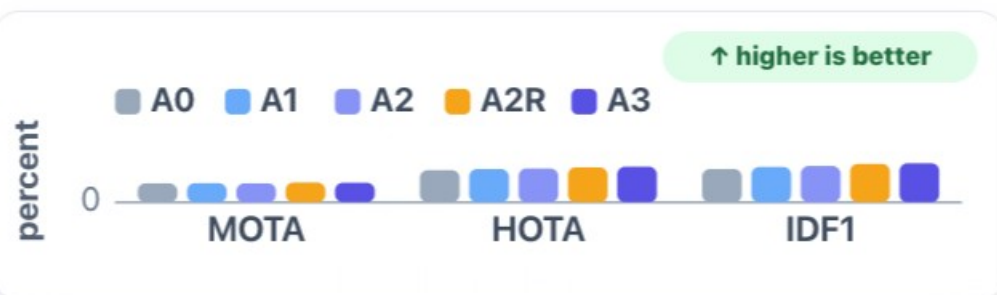
Each label has a score. A score of 0 means “ignore this area”, so we remove it. We keep labels with score 1.

Occlusion: the object is hidden by another object. · **Truncation:** part of the object is outside the image.

Ablation Study: What Each Part of AC-MOT Adds

THE MAIN IDEA

We switch on AC-MOT (Adaptive Control for Multi-Object Tracking) **one part at a time** on the same videos, so every change in the score can be traced to **that part**.



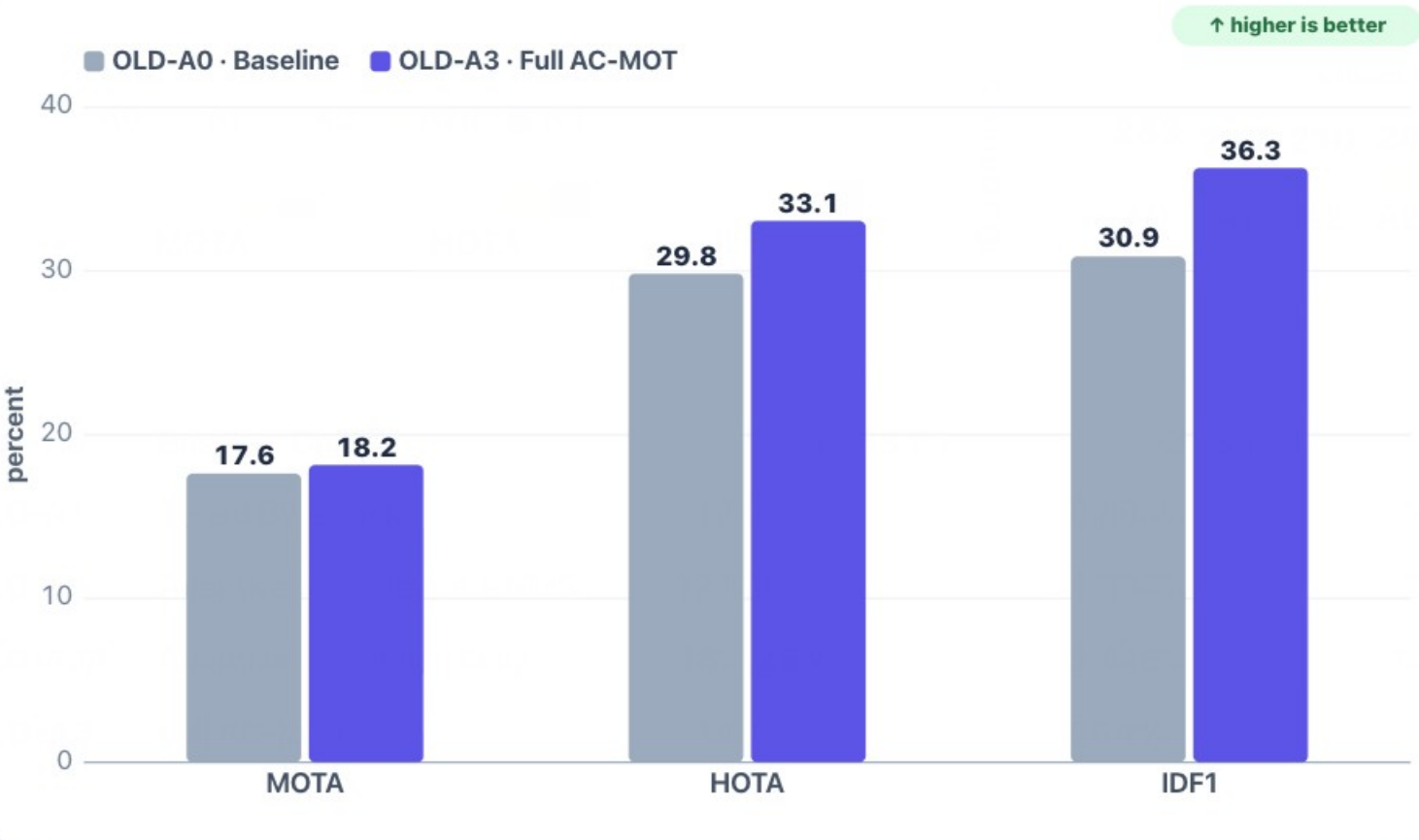
STAGE	CONFIGURATION	MOTA ↑	HOTA ↑	IDF1 ↑	IDS ↓	FPS ↑
OLD-A0	Baseline Default	17.633%	29.837%	30.892%	283	44.09
OLD-A1	Tuned ByteTrack	17.802% +1.0%	30.864% +3.4%	32.854% +6.4%	217 -23.3%	44.51 +1.0%
OLD-A2	Adaptive Confidence + NMS	17.636% +0.02%	31.384% +5.2%	33.727% +9.2%	210 -25.8%	40.65 -7.8%
OLD-A2R	Adaptive Resolution Only	18.425% +4.5%	32.446% +8.7%	35.669% +15.5%	241 -14.8%	39.56 -10.3%
OLD-A3	Full AC-MOT	18.165% +3.0%	33.064% +10.8%	36.296% +17.5%	271 -4.2%	42.085 -4.55%

Small numbers = change compared with **OLD-A0 (baseline)**, in percent of its value · **green = better** · **red = worse**

Baseline vs Full AC-MOT: What Went Up

THE MAIN IDEA

The full AC-MOT (Adaptive Control for Multi-Object Tracking) improved **all three quality scores** over the baseline — most of all **IDF1** (Identity F1 score) (+5.404) and **HOTA** (Higher Order Tracking Accuracy) (+3.227) — at the cost of some **speed**.



METRIC	OLD-A0 · BASELINE	OLD-A3 · FULL AC-MOT	CHANGE
MOTA ↑	17.633%	18.165%	+0.532
HOTA ↑	29.837%	33.064%	+3.227
IDF1 ↑	30.892%	36.296%	+5.404
IDS (identity switches) ↓	283	271	4.2% fewer
FPS ↑	44.09	42.085	-2.005

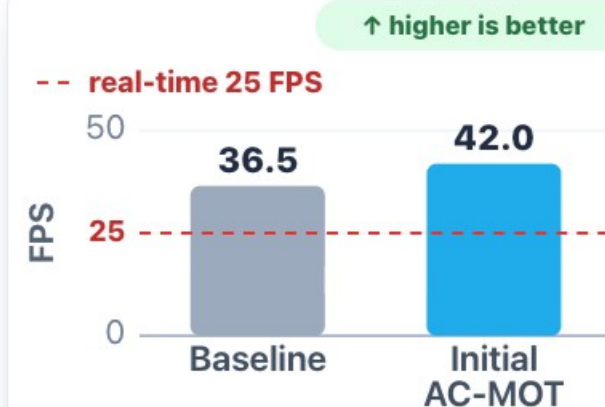
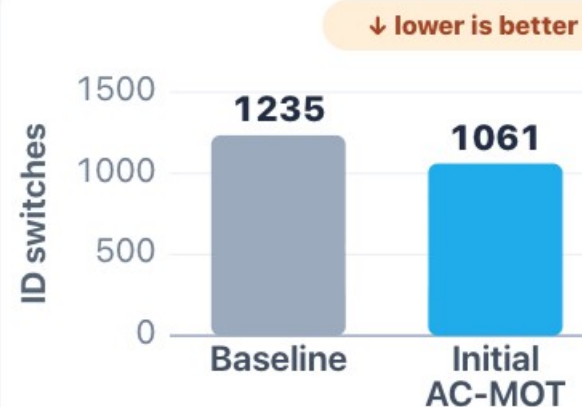
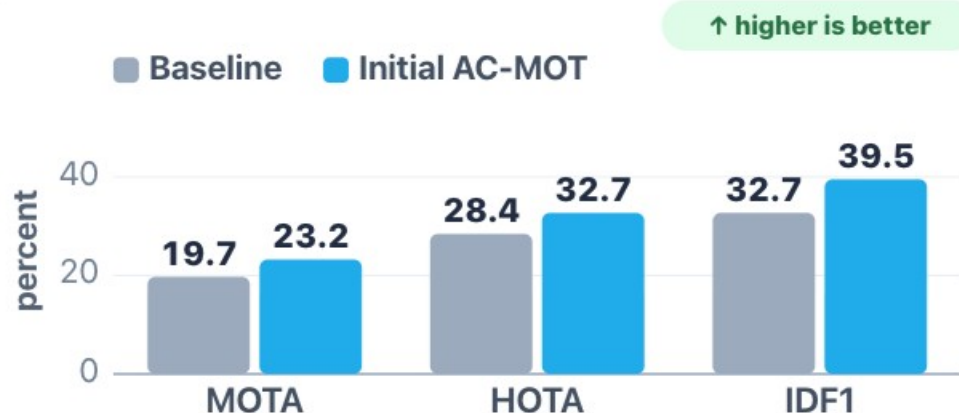
What went up
Keeping the **same identity** (IDF1, HOTA) improved the most; MOTA (Multiple Object Tracking Accuracy) rose a little and there were **4.2% fewer ID switches**.

The cost
Speed dropped from **44.09 to 42.085 FPS** (frames per second) — still above the 25 FPS real-time line.

Old AC-MOT on the Test Set

THE MAIN IDEA

After the ablation on **validation**, the full old AC-MOT (Adaptive Control for Multi-Object Tracking) was run **once on the test set**. It beat the baseline on every metric.



SYSTEM	MOTA ↑	HOTA ↑	IDF1 ↑	IDS ↓	FPS ↑
Baseline	19.729	28.430	32.724	1235	36.53
Old AC-MOT	23.236	32.698	39.516	1061	41.96
Change	+3.507	+4.268	+6.792	14.1% fewer	+5.43

Same Frames, Same Detector, Different Control

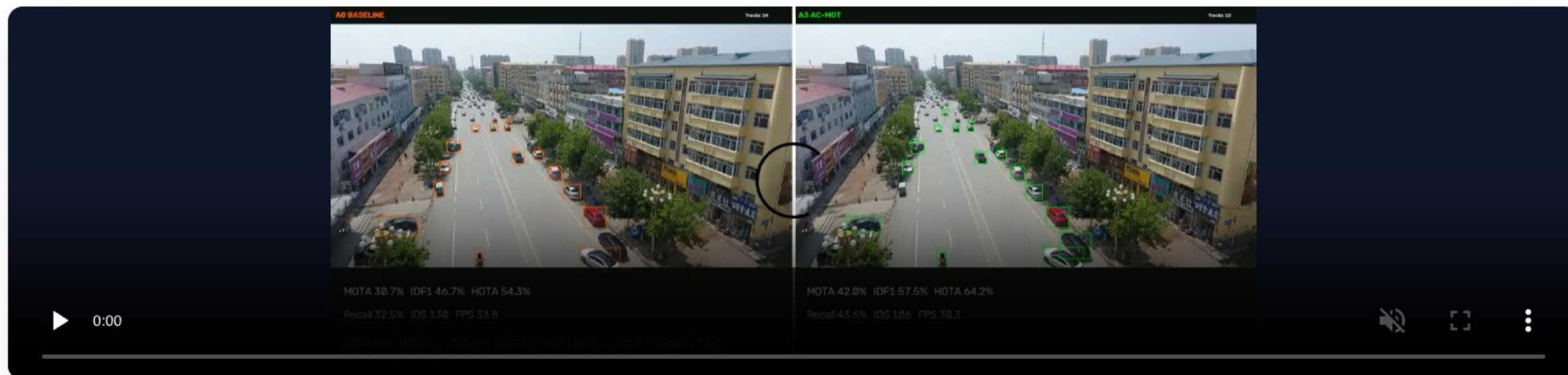
THE MAIN IDEA

The same video frames with the same detector, **without and with our control**. Watch the hard parts: our system **keeps more objects tracked**.

Clip 1 · uav0000249

Clip 2 · uav0000188 (tiny objects)

Clip 3 · uav0000249 · final system



Same frames, same detector, different control. AC-MOT (Adaptive Control for Multi-Object Tracking) keeps more objects tracked in hard frames. **Videos show the idea; the numbers are the evidence** (on-screen ID markers are a visual aid, not official IDS).

VI SECTION VI

V1: Letting Validation Data Choose

The idea worked — but were the hand-picked weights the best? Optuna with TPE (Tree-structured Parzen Estimator) searched on validation data only.

Why Crowd = 0.30? Why Edge = 0.20?

THE MAIN IDEA

The five cues have a reason. The **exact weights did not** — we chose them **by hand**.

CUE	HAND-SET WEIGHT	PROOF IT IS THE BEST?
Crowd	0.30	?
Tiny objects	0.30	?
Edge complexity	0.20	?
Low light	0.10	?
Blur	0.05	?

Has a scientific reason

The **five cues**: crowd, tiny objects, edges, low light and blur.

Chosen by hand

The **exact weights** 0.30 / 0.30 / 0.20 / 0.10 / 0.05.

So: instead of choosing the weights by hand, **let optimization choose them from validation data**.

Before Optuna: We First Screened the Operating Settings

THE MAIN IDEA

V1 did not start with an arbitrary search space. We first tested reasonable operating values on the **7 validation videos**, then froze the smaller search space for Optuna.

SCREENING STAGE	WHAT WE TESTED	WHAT ENTERED OPTUNA
Input size	15 values: 512 → 960, step 32	512, 928, 960
Confidence	10 values: 0.05 → 0.50	0.25 → 0.45
NMS IoU (Intersection over Union)	11 values: 0.30 → 0.80	0.30 → 0.70
Temporal settings	25 combinations	window 7, stride 10

61 screening runs

15 resolution + 10 confidence + 11 NMS (Non-Maximum Suppression) + 25 time runs.

Then 50 Optuna trials

Optuna searched the frozen space for the V1 settings.

Validation only

The test set was kept separate for the final judgment.

Every Screening Point Was a Full Pipeline Run

THE MAIN IDEA

We did not judge a value by itself. We changed one setting, kept the rest fixed, ran the full system on the **7 validation videos**, and measured the tracking result.



Example: input-size sweep

Keep (YOLOv8n), (ByteTrack), (Confidence), (NMS IoU) and time settings fixed.

Run 512, then 544, then 576, and continue through 928 and 960.

Compare quality with speed: does a larger image improve tracking, and how much (FPS = frames per second) is lost?

The same rule each time

For (Confidence), change only the confidence value.

For (NMS IoU), change only the NMS value.

For time settings, change one (Window, Stride) pair at a time.

One row in a sweep file means: **one complete run** → **seven validation sequences** → **final tracking output** → **TrackEval metrics**.

Why Window = 7 and Stride = 10?

THE MAIN IDEA

We tested every pair from two fixed lists before V1. The best **valid** pair was selected from the validation results, then frozen: **window = 7** and **stride = 10**.

All values we tried

Smoothing window 1, 3, 5, 7, 9 readings

Analysis stride 1, 5, 10, 15, 20 frames

Total sweep $5 \times 5 = 25$ pairs

Every window was paired with every stride, from (1,1) through (9,20).
Every pair used the same validation setup.

How the sweep chose (7, 10)

25 pairs

run on
validation



Filter

$\text{FPS} \geq 25 \cdot \text{IDS} \leq 271$



Choose

highest MOTA

The selected pair: **window 7, stride 10**

MOTA (Multiple Object Tracking Accuracy) 18.1647 · HOTA (Higher Order Tracking Accuracy) 33.0640 · IDF1 (Identity F1 score) 36.2962 · IDS (identity switches) 271 · FPS (frames per second) 43.9472

If two choices were equal, we used lower IDS, then higher HOTA, IDF1 and FPS.

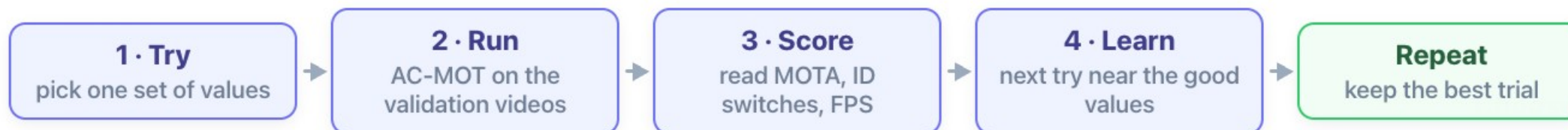
Separate stages: 25 time pairs + 50 Optuna trials = 75 validation runs. **Optuna Trial 24 is not run 49 of 75.**

Correct order: **25 time pairs** → **select and freeze (7,10)** → **freeze the Optuna search space** → **50 V1 Optuna trials** → **select Optuna Trial 24**.

Why Optuna Was Introduced

THE MAIN IDEA

Our first design used **numbers we picked by hand**. **Optuna** is a search tool that **tries many sets of numbers for us** and keeps the set that works best on the **validation videos**.



Before: chosen by hand

- how much each of the five cues counts (the SCI weights)
- where a scene stops being easy and becomes hard
- the detector settings for each kind of scene

We could not be sure that these guesses were good.

After: chosen by the results

- the **validation videos** decide — not our guesses
- it tests **many combinations** — more than a person can try by hand
- the **test set is never used**, so the final test stays fair

THE TOOL

Optuna with the **TPE** sampler. It is a **search method, not an AI model**: it trains nothing — it only decides which values to try next, based on the scores so far.

What Optuna Changed in V1 — and What It Did Not

THE MAIN IDEA

Optuna did **not** change **how we measure the scene**. It only chose **how AC-MOT** (Adaptive Control for Multi-Object Tracking) **reacts** to what it measures. Each trial tested **one full set** of these choices — one complete version of AC-MOT.

Fixed · how we measure the scene

crowd	number of boxes $\div$ 30
tiny objects	share of boxes smaller than 32×32 px
busy edges	edge density $\div$ 0.14
dark	brightness below 80
blurry	sharpness (Laplacian variance) below 180
smoothing	average of the last 7 readings
how often	every 10 frames

Chosen by Optuna · how AC-MOT reacts

SCI (Scene Complexity Index) weights	how much each cue counts: crowd · tiny · edges · dark · blur
scene limits	where SCI turns a scene from easy to medium to hard
detector settings	the confidence and NMS (Non-Maximum Suppression) IoU (Intersection over Union) for easy and for hard scenes

All of these are chosen **together**. One trial = one full set = **one complete AC-MOT**, tested on the validation videos.

What Happens in One Optuna Trial

THE MAIN IDEA

One trial = **run the complete pipeline** on the validation videos with one configuration, then **score it**. TPE (Tree-structured Parzen Estimator) uses earlier trials to steer later trials toward **more promising regions**.

Placeholder for the illustration of one V1 trial.

V1 Selection on the Validation Set

THE MAIN IDEA

Trial 24 was selected by the **V1 validation-selection rules**, and then frozen before the test-set run.

MOTA

23.038HOTA (HIGHER ORDER
TRACKING ACCURACY)**36.110**IDF1 (IDENTITY F1
SCORE)**40.758**

IDS

270

FPS

37.17

V1 selection rules

- FPS (frames per second) must stay acceptable (at least 25)
- IDS (identity switches) must not exceed the reference: **271** (Old-A3, the initial full AC-MOT on validation)
- then take the **highest MOTA** (Multiple Object Tracking Accuracy)
- **50 trials** · TPE (Tree-structured Parzen Estimator) sampler

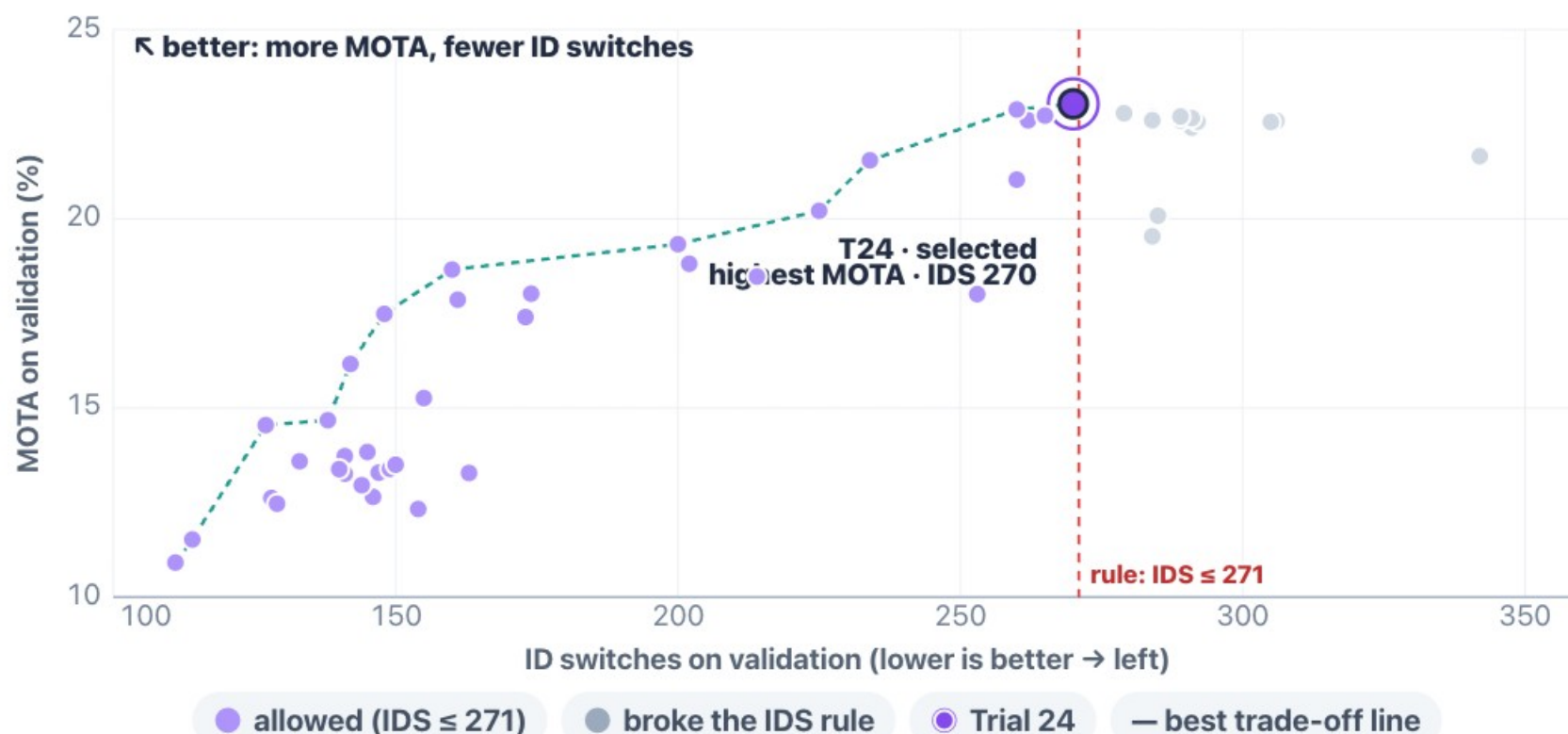
Validation, not test

These numbers come from the **validation split** used for selection. The test-set comparison follows on the next slides.

All V1 Trials: The Trade-off Curve

THE MAIN IDEA

Each dot is **one V1 trial** on the validation videos. Dots **right of the red line** broke the ID-switch rule. **Trial 24** has the highest MOTA (Multiple Object Tracking Accuracy) — of the allowed trials and of **all 50**.



COMPLETED TRIALS

50

trial 33 did not finish

ALLOWED BY THE RULES

37

13 broke IDS ≤ 271

SPEED RULE

all ≥ 25

slowest trial: 34.49 FPS

Trial 24: MOTA 23.038 · IDS (identity switches) 270 · FPS (frames per second) 37.17

Why Trial 24? The Selection Rules on a Chart

THE MAIN IDEA

A trial could only win if it was **fast enough** and did **not add ID switches**. Among the trials that passed both rules, **Trial 24 had the highest MOTA** (Multiple Object Tracking Accuracy).

RULE 1 · FPS MUST STAY AT LEAST 25

↑ higher is better

-- rule: at least 25 FPS



✓ 37.17 FPS (frames per second)

RULE 2 · IDS (IDENTITY SWITCHES) MUST NOT EXCEED OLD-A3 (271)

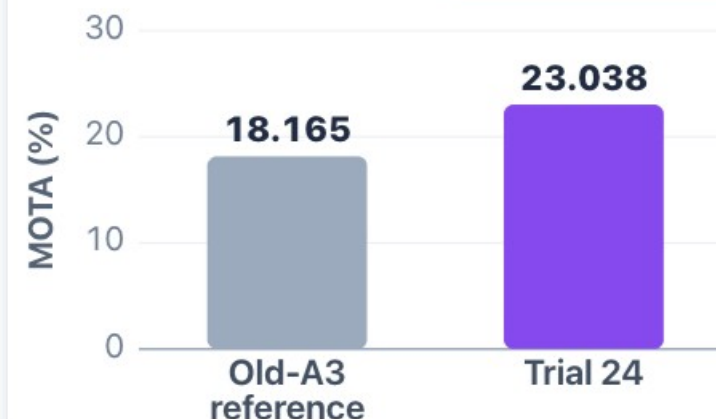
↓ lower is better



✓ $270 \leq 271$

THEN · THE HIGHEST MOTA WINS

↑ higher is better



✓ 23.038 vs 18.165

If two trials had the same MOTA, the tie would go to fewer ID switches, then higher HOTA (Higher Order Tracking Accuracy), IDF1 (Identity F1 score) and FPS. Everything was decided on **validation** — the test set was never used.

QUICK QUESTION



Which Cue Did the Search Trust Most?

Crowd, tiny objects, edges, night or blur? Our hand-set guess gave crowd and tiny the most weight. Make a guess — the answer is coming.

V1 Trial 24: The Selected SCI Weights

THE MAIN IDEA

The selected V1 configuration trusts **edge complexity most** and **low light least** — very different from the initial hand-designed weights.

Original
(hand-set)



V1 Trial 24
(learned)



■ Crowd ■ Tiny objects ■ Edge complexity ■ Night ■ Blur

CUE	INITIAL	V1 TRIAL 24
Crowd	0.30	0.129
Tiny	0.30	0.222
Edge	0.20	0.434
Night	0.10	0.054
Blur	0.05	0.161

The optimizer says: **edge complexity matters much more** than we assumed by hand (0.20 → 0.434).

V1 Trial 24: Easy, Medium and Hard Scenes

THE MAIN IDEA

The SCI (Scene Complexity Index) ranges were **not set by hand after looking at SCI values**. They **emerged from the joint Optuna optimization**, together with the detector parameters.



REGIME	SCI RANGE	DETECTOR PARAMETER	VALUE
Easy	$SCI < 0.13535$	conf_easy	0.30
Medium	$0.13535 \leq SCI < 0.28729$	conf_hard	0.40
Hard	$SCI \geq 0.28729$	nms_easy	0.35
threshold_mid	0.13534938199219218	nms_hard	0.35
threshold_high	0.28728676236279177		

Validation Builds It, Test Judges It

THE MAIN IDEA

Validation videos build the system; test videos judge it. Everything is chosen on 7 validation videos; the 17 test videos are used **once**, at the end.

Validation · 7 sequences

All 50 V1 Optuna trials run here. Every setting is chosen and frozen here.

Test-dev · 17 sequences

Frozen systems, run once: the Baseline and V1 (Trial 24) — and later V2. No search, no tuning, no second try.

Same GPU (graphics processing unit), same tool

Tesla T4 for all, TrackEval pinned to one version, seed 42.

What we do not mix

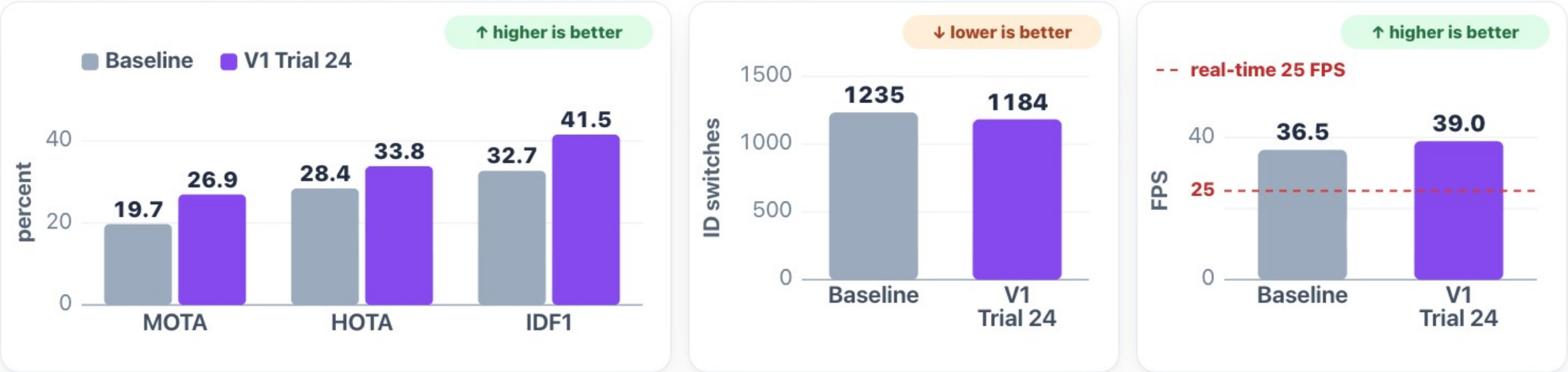
Validation and test-set numbers come from different runs, so they are never put in one table.

In one line: validation = learn, compare, choose, freeze. Test = judge the frozen system once.

V1 on the Test Set: The Strongest Quality Result

THE MAIN IDEA

After freezing V1, we tested it **once**. It gave our **biggest quality gain** over the baseline.



SYSTEM	MOTA ↑	HOTA ↑	IDF1 ↑	IDS ↓	FPS ↑
Baseline	19.729	28.430	32.724	1235	36.53
V1	26.948	33.835	41.546	1184	38.98
V1 – Baseline	+7.220 pp	+5.405	+8.822	–4.1%	+2.46

Where Is the Problem? ID Switches

THE MAIN IDEA

V1 has the best quality, but it cut ID switches by **only 4.1%**: 1235 → 1184.



IN THE V1 SEARCH

ROLE

MOTA (Multiple Object Tracking Accuracy)

goal — as high as possible

IDS

rule — only stay ≤ 271

FPS (frames per second)

rule — only stay acceptable

IDS (identity switches) was a **constraint**, not an **objective**. Optuna only needed IDS to **pass the rule** — not to make it as low as possible.

So identity stability did **not** improve as much as tracking quality. This is the second research question.

VII

SECTION VII

V2: Two Goals at Once

Maximize MOTA (Multiple Object Tracking Accuracy) and minimize ID switches together — and find the trade-off between them.



Why V2 Was Created

THE MAIN IDEA

V1 improved quality strongly, but **identity switches were not minimized**. So a **second optimization direction** was investigated.

V1 · ONE OBJECTIVE WITH RULES

Highest MOTA (Multiple Object Tracking Accuracy), while FPS (frames per second) ≥ 25 and IDS ≤ 271 .

V2 · MULTI-OBJECTIVE

- maximize MOTA
- minimize IDS
- with an FPS constraint

V2 was **not meant to replace V1** everywhere. It is **another operating point** on the quality / identity / efficiency trade-off.

Two Goals: What Is a Trade-off?

THE MAIN IDEA

With two goals, one setting can be better in one goal and **worse in the other**. Neither one wins everything.

TRIAL A

MOTA (Multiple Object Tracking Accuracy)	high
ID switches	high

TRIAL B

MOTA	lower
ID switches	very low

This is a **trade-off**. All the best trade-off points together form the **Pareto front**.

The Trade-off Curve (Pareto Front)

WHAT IT MEANS

Each dot is a setting that **cannot improve one goal without losing on the other**. Top-left is best; we picked the **balanced point**, Trial 22.

What a Pareto point is

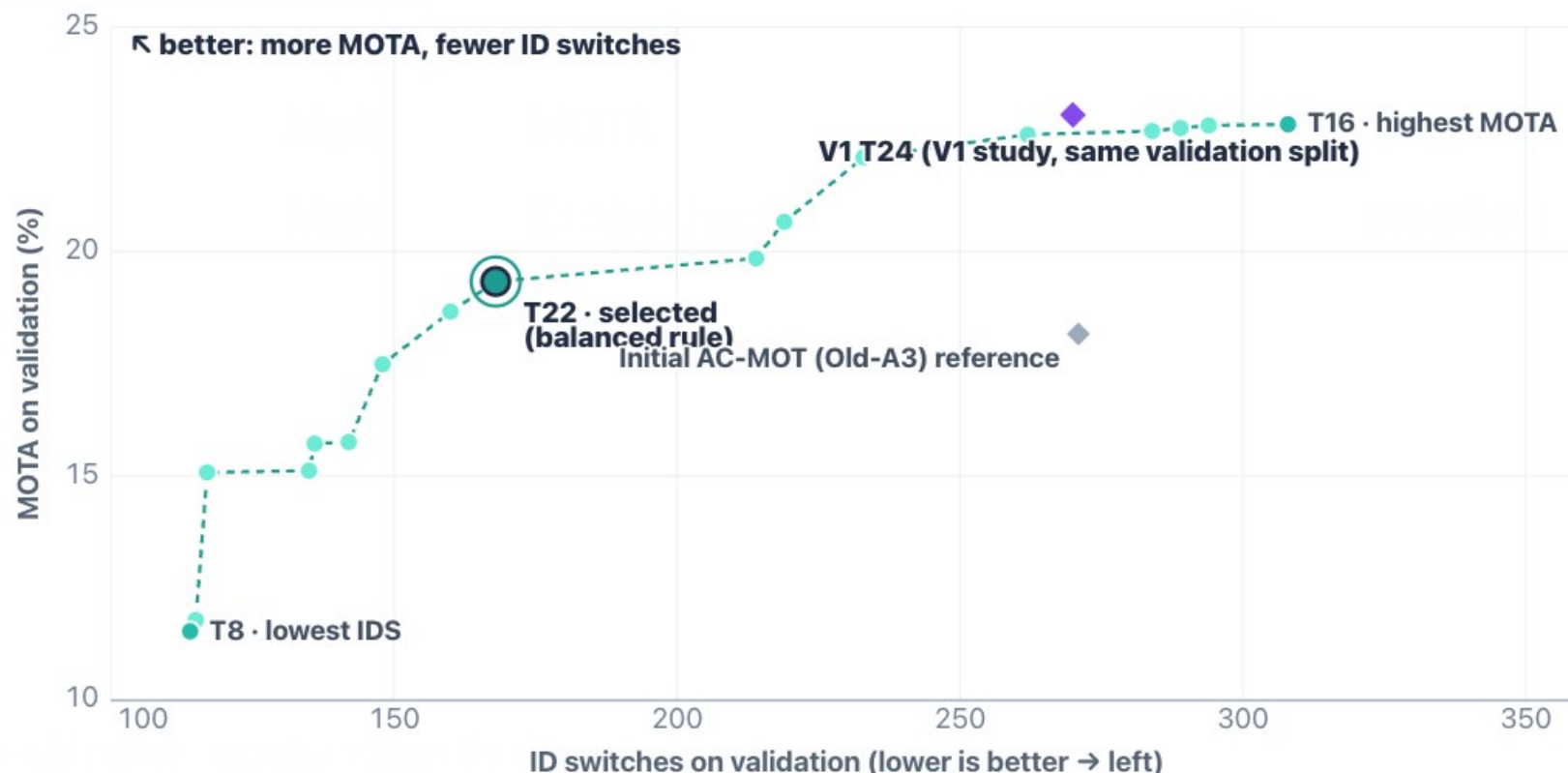
A setting where you cannot get more MOTA (Multiple Object Tracking Accuracy) without getting more ID switches.

How to read it

- Up = more MOTA
- Left = fewer ID switches
- Best area: top-left

The pick, fixed in advance

Give equal weight to both goals → **Trial 22**.



V2 Search: 49 Trials, One Balanced Pick

THE MAIN IDEA

The V2 search ran on **validation only**. **49 of 50** planned trials finished, and a **fixed balanced rule** picked Trial 22.

PLANNED

50

trials

COMPLETED

49

Trial 0 → Trial 48

SELECTED

Trial 22

balanced selection

V2 goals

- **maximize** MOTA (Multiple Object Tracking Accuracy)
- **minimize** IDS (identity switches)
- FPS (frames per second) must stay at least 25

Balanced selection

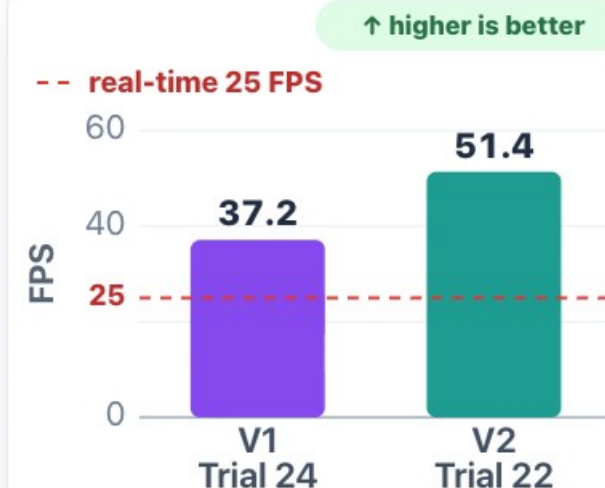
50% MOTA + 50% IDS qualityApplied to the Pareto solutions → **Trial 22**.

Always say it correctly: 50 trials were planned; **49 completed**.

V1 vs V2 on Validation

THE MAIN IDEA

Now the two versions have **different characters**: **V1 = quality-oriented** · **V2 = identity + speed-oriented**.

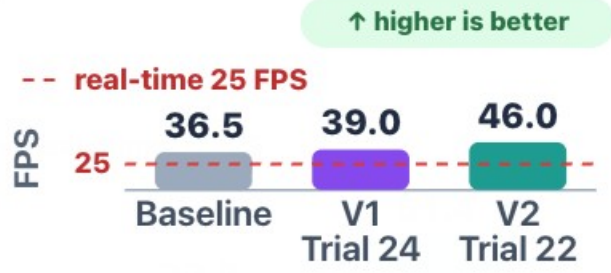
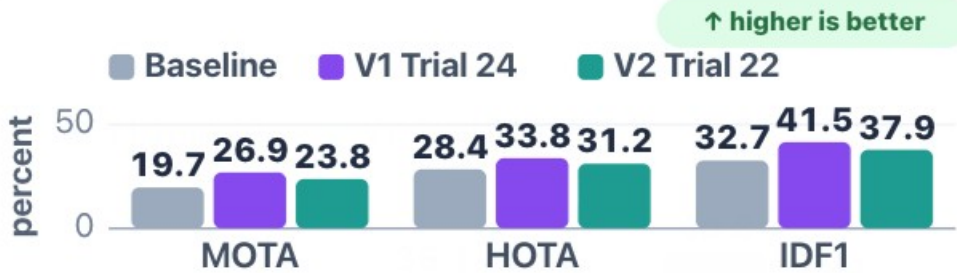


SYSTEM	MOTA ↑	HOTA ↑	IDF1 ↑	IDS ↓	FPS ↑
V1 · Trial 24	23.038	36.110	40.758	270	37.17
V2 · Trial 22	19.330	31.651	34.226	168	51.41

V2 on the Test Set

THE MAIN IDEA

V2 is not better than V1. It is a different point on the trade-off.



SYSTEM	MOTA ↑	HOTA ↑	IDF1 ↑	IDS ↓	FPS ↑
Baseline	19.729	28.430	32.724	1235	36.53
V1	26.948	33.835	41.546	1184	38.98
V2	23.792	31.218	37.870	919	46.02

V2 lost vs V1: MOTA (Multiple Object Tracking Accuracy) −3.157 pp · HOTA (Higher Order Tracking Accuracy) −2.617 · IDF1 (Identity F1 score) −3.676

V2 gained vs V1: IDS (identity switches) 1184 → 919 (22.4% fewer) · FPS (frames per second) +7.04 · 28.4% fewer false positives

So Which One Is Better — V1 or V2?

THE MAIN IDEA

There is **no absolute best**. The right version depends on **the priority**.

IF THE PRIORITY IS
the **highest tracking quality**

→ **V1**

IF THE PRIORITY IS
fewer ID switches, fewer false positives and higher speed

→ **V2**

Showing this trade-off clearly is itself a **scientific result** of the thesis.

VIII SECTION VIII

Is the Gain Real?

A paired bootstrap on the test videos, then a zero-tuning test on a completely different dataset: UAVDT (UAV Detection and Tracking benchmark).

What Is a Paired Bootstrap — and Why We Used It

THE MAIN IDEA

A bootstrap checks whether a gain is **real** or just **luck with the test videos we happened to have**.

Why?

The test set has only **17 videos**. If a few of them are easy, the gain could look **bigger than it really is**.

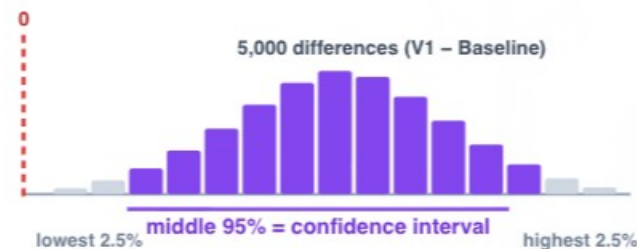
How to read it

Interval **fully above 0** → the gain is real.

Interval **crosses 0** → we cannot claim a gain.

How it is measured

- 1 Pick **17 videos at random** from the 17 test videos — the same video may be picked twice.
- 2 On **these same videos**, score V1 and the Baseline and take the **difference**. Same videos for both = **paired**.
- 3 Repeat **5,000 times** → 5,000 differences.
- 4 Cut off the lowest and highest 2.5% → the **middle 95%** is the confidence interval.



drawing · shape only, no real numbers

Paired Bootstrap: Is the V1 Gain Real?

THE MAIN IDEA

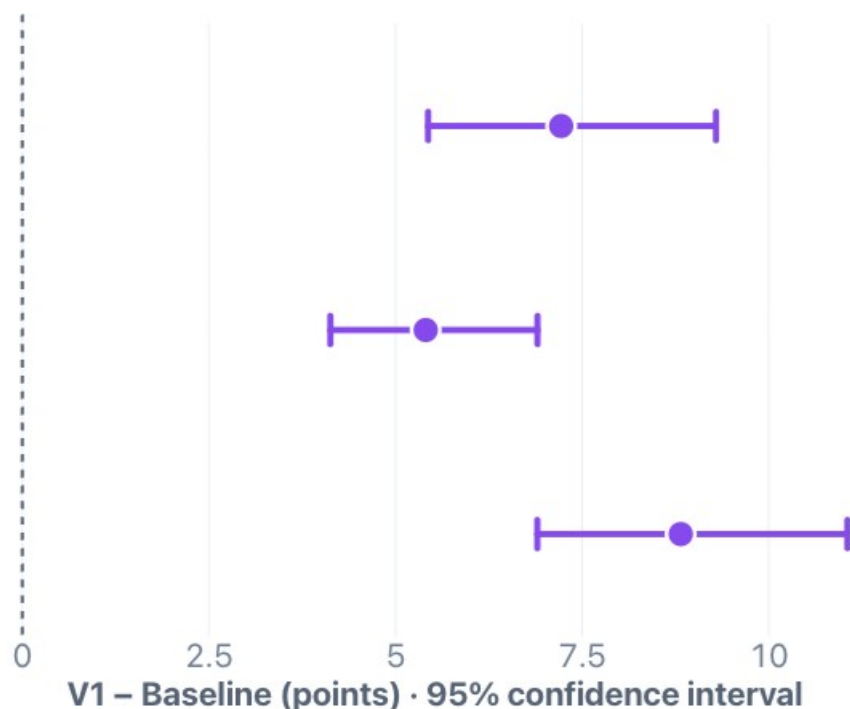
Maybe a few easy videos made the gain look big? We **resampled the test videos 5,000 times**. The V1 quality gains stay **clearly above zero**.

pick test videos at random, with repeats → compute V1 – Baseline → repeat 5,000 times → middle 95% = confidence interval (CI)

MOTA

HOTA

IDF1



V1 – BASELINE

GAIN

95% CI

MOTA (Multiple Object Tracking Accuracy)

+7.2195

[5.4362, 9.2934]

HOTA (Higher Order Tracking Accuracy)

+5.4051

[4.1273, 6.9022]

IDF1 (Identity F1 score)

+8.8220

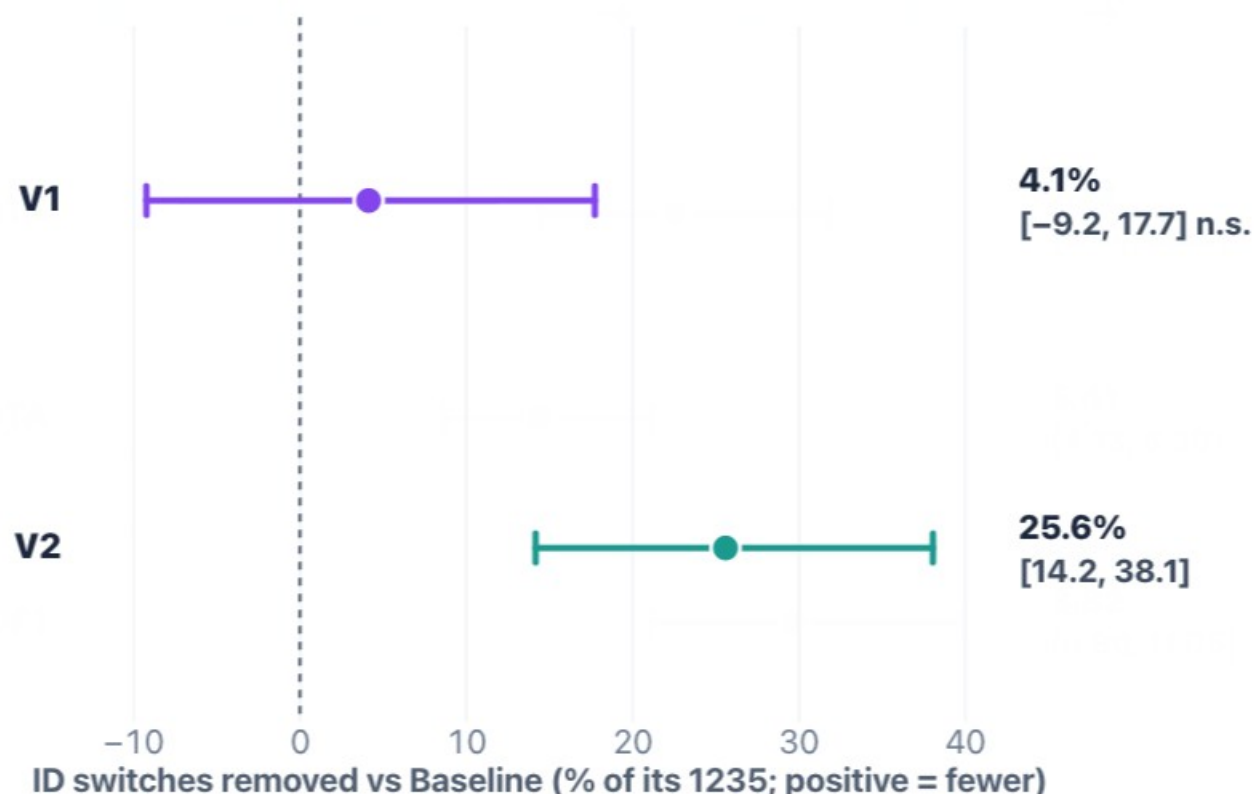
[6.9005, 11.0537]

All three intervals are **above zero** → the quality gain is **statistically strong**.

ID Switches: V1 Not Significant, V2 Significant

THE MAIN IDEA

For V1, the ID-switch interval **crosses zero** — we cannot claim a real improvement. For V2 it does not. **This is exactly why V2 was needed.**



VS BASELINE	IDS REMOVED	95% CI	REAL?
V1	4.1%	[-9.2%, 17.7%]	no
V2	25.6%	[14.2%, 38.1%]	yes

Percent of the baseline's 1235 ID switches on the test set.

We **cannot** say V1 made a significant ID-switch improvement. V2 did.

Did It Only Work Because We Tuned on VisDrone?

THE MAIN IDEA

To check, we **froze V1 and V2** and ran them on a **completely different dataset: UAVDT** (UAV Detection and Tracking benchmark).



What we did NOT do

- no training
- no Optuna
- no retuning
- no recalibration

What this tests

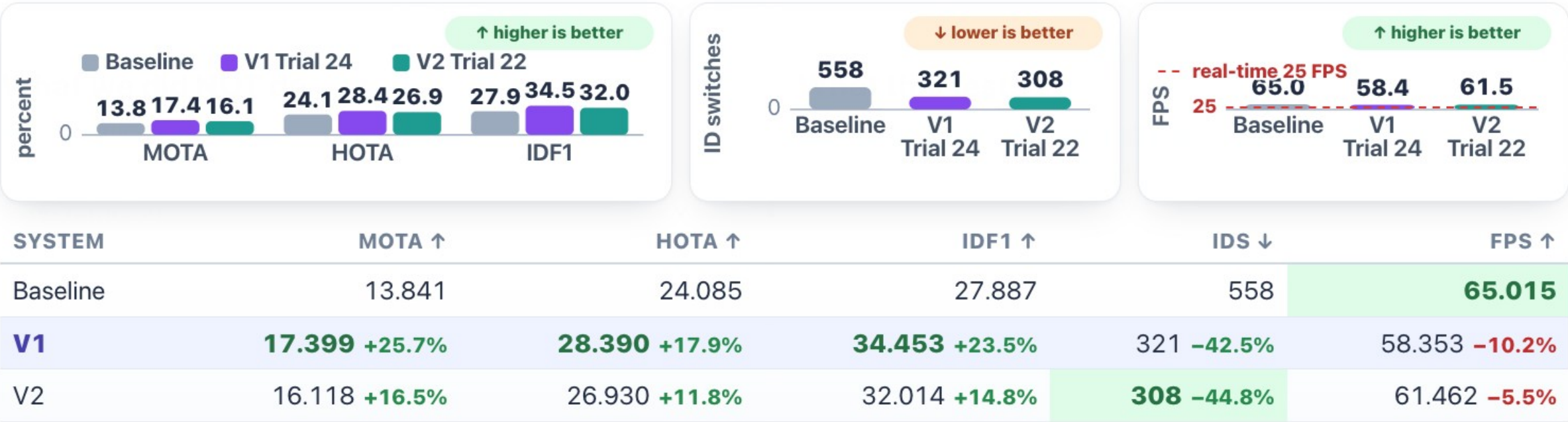
Zero-tuning external generalization

Does the gain survive on data the system has never seen?

Cross-Dataset Evidence: UAVDT

THE MAIN IDEA

The improvement transferred to UAVDT (UAV Detection and Tracking benchmark) **without tuning**.
V1 vs Baseline: MOTA (Multiple Object Tracking Accuracy) **+3.558** · HOTA (Higher Order Tracking Accuracy) **+4.305** · IDF1 (Identity F1 score) **+6.565** · IDS (identity switches) **−42.5%**. The bootstrap confirmed these main gains are **significant**.



Small numbers = change compared with the Baseline on UAVDT, in percent of its value · green = better · red = worse

IX

SECTION IX

Transfer to Published U2MOT

Can the AC-MOT (Adaptive Control for Multi-Object Tracking) control layer work with a different published detector and tracker pipeline?

First, We Reproduced the Published U2MOT Pipeline

THE MAIN IDEA

We started with the published **U2MOT** method, without AC-MOT (Adaptive Control for Multi-Object Tracking), and ran it on the same 17 test-dev sequences.

Pipeline and data

Detector: **YOLOX-X**

Tracker: **U2MOT**

VisDrone2019-MOT test-dev

17 sequences · 6,635 frames

Code changes were only compatibility fixes, not model tuning.

Our reproduced U2MOT baseline

MOTA (Multiple Object Tracking Accuracy)	53.900
IDF1 (Identity F1 score)	69.800
IDS (identity switches)	1,239
FP (false positive)	41,385
FN (false negative)	63,241

This gives us a fixed reference: **our independent reproduction of U2MOT without AC-MOT.**

Important: These values are not copied from the ICCV (International Conference on Computer Vision) 2023 paper. They come from an independent reproduction under our project evaluation protocol. Small differences can come from the implementation version, checkpoint, preprocessing, runtime settings, class filtering or evaluation protocol.

SCI Had to Be Recalibrated for the New Pipeline

THE MAIN IDEA

Weights define how we calculate SCI (Scene Complexity Index). Boundaries define how we label the result: Easy, Medium or Hard. We kept the V1 weights and changed only the boundaries for the new U2MOT range.

What stayed fixed

V1 weights: **0.1295 / 0.2217 / 0.4337 / 0.0536 / 0.1615**

Window = **7** · Stride = **10**

No new Optuna search was run on U2MOT.

Validation SCI range

Minimum **0.34155** · Mean **0.55452**

Median **0.54897** · Maximum **0.72456**

SCI VALUE	SCENE LABEL	HOW THE BOUNDARY WAS SET
SCI < 0.5185453	Easy	Lower third of validation SCI values
0.5185453 ≤ SCI < 0.5895548	Medium	Middle third of validation SCI values
SCI ≥ 0.5895548	Hard	Upper third of validation SCI values

How we got the limits: calculate SCI at every validation analysis point, sort the values, then use about the **33rd percentile** and **67th percentile**. The old V1 limits **0.13535 / 0.28729** would label nearly all U2MOT points as Hard because the new minimum is **0.34155**.

AC-MOT Changed the Detector Settings by Scene Difficulty

THE MAIN IDEA

The U2MOT tracker settings stayed fixed. AC-MOT (Adaptive Control for Multi-Object Tracking) changed only the detector settings for easy, medium and hard scenes.

SCENE	CONFIDENCE	NMS IOU	INPUT SIZE
Easy	0.15	0.70	1280 × 704
Medium	0.12	0.65	1440 × 800
Hard	0.09	0.60	1600 × 896

Tracker settings stayed fixed

Track high = 0.5 · Track low = 0.1 · Match = 0.8 · Buffer = 15

Easy scenes use a smaller input. Hard scenes use the full input and a lower confidence limit.

On Validation, AC-MOT Improved Quality and Identity

THE MAIN IDEA

On the 7 validation sequences, AC-MOT (Adaptive Control for Multi-Object Tracking) improved **MOTA** (Multiple Object Tracking Accuracy), **IDF1** (Identity F1 score), **IDS** and **FP**. The number of missed objects increased.

METRIC	U2MOT BASELINE	U2MOT + AC-MOT	CHANGE
MOTA	75.1273	75.4738	+0.3465
IDF1	75.5578	76.5775	+1.0196
IDS	1,947	1,908	-39
FP	8,837	7,992	-845
FN	7,091	7,726	+635

Validation shows that the control layer can change the operating point of a different pipeline.

We Checked Speed Fairly, Then Locked the Setup

THE MAIN IDEA

Both systems used the same dynamic runtime code, and all controller choices were frozen before the final test.

Matched runtime

Fixed U2MOT: **3.707 FPS** (frames per second)

U2MOT + AC-MOT (Adaptive Control for Multi-Object Tracking): **4.663 FPS**

Relative change: **+25.76%**

AC-MOT uses smaller inputs in easier scenes.

Frozen before test-dev

SCI (Scene Complexity Index) weights and limits

Easy / Medium / Hard actions

Tracker settings, code commit and checkpoint

No test-dev tuning and no changes after the freeze.

The final test was run only after the controller and all settings were locked.

Final Test: Quality Stayed Close, IDs and FP Fell

THE MAIN IDEA

AC-MOT (Adaptive Control for Multi-Object Tracking) did not beat U2MOT on every metric. It kept quality close while reducing identity switches and false positives.

METRIC	U2MOT BASELINE	U2MOT + AC-MOT	CHANGE
MOTA (Multiple Object Tracking Accuracy)	53.9000	53.7668	-0.1332
IDF1 (Identity F1 score)	69.8000	69.8494	+0.0494
IDS	1,239	1,152	-87
FP	41,385	40,155	-1,230
FN	63,241	64,801	+1,560

Same-method speed

Final AC-MOT: 4.082 FPS (frames per second) on 6,635 frames.

Correct claim

Quality was approximately preserved while IDS and FP were reduced.

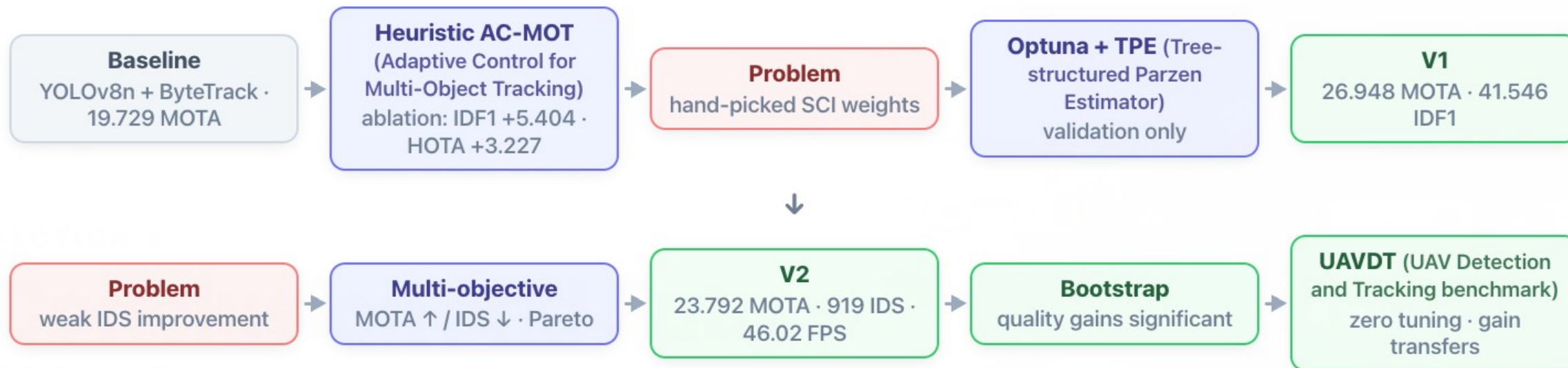
X

SECTION X

Conclusion and Future Work

The whole story in one line, the contributions, and what comes next.

The Whole Story in One Line



Every step answered **one question** — and opened the next one.

Main Contributions

1 A lightweight **Scene Complexity Index** built from **five low-cost cues**.

2 **Adaptive detector-side control** according to scene difficulty.

3 **Validation-driven joint optimization** of SCI weights, regime boundaries and detector operating parameters.

We measured the balance between **tracking quality, identity stability and speed**.

4 We checked the result with a paired bootstrap and a UAVDT (UAV Detection and Tracking benchmark) test with no new tuning.

Conclusion

IN ONE SENTENCE

"AC-MOT (Adaptive Control for Multi-Object Tracking) **does not replace** the detector or tracker. It adds a **scene-aware control layer** that changes the detector operating point according to scene complexity."

Fixed detector settings	→ ignore temporal scene variation
SCI (Scene Complexity Index)	→ estimates scene complexity
Adaptive controller	→ selects detector operating parameters
V1	→ prioritizes tracking quality
V2	→ prioritizes identity and efficiency trade-offs
UAVDT (UAV Detection and Tracking benchmark)	→ supports cross-dataset transfer

Directions for Future Work

THE MAIN IDEA

Every next step targets a **weakness we measured**: camera motion, a smarter input size, better appearance matching, and other detectors.

1 · Camera-motion compensation

The clearest next gain. It targets exactly the ID switches that adaptive input size adds.

2 · Density-gated input size

Raise the input size only when many tiny objects are present, not on SCI (Scene Complexity Index) alone.

3 · A proper deep ReID (re-identification)

Replace the colour histogram with a small deep appearance model that stays real-time.

4 · SCI beyond YOLO (You Only Look Once)

Drive RT-DETR (Real-Time DEtection TRansformer) or DINO with the same signal, and test on AU-AIR.

Note: these ideas come from what we measured, not from guesses.

AC-MOT (ADAPTIVE CONTROL FOR MULTI-OBJECT TRACKING)
— **ADAPTIVE SCENE CALIBRATION FOR REAL-TIME DRONE**
TRACKING

Thank you

Questions?

Ahmed Gouda Ismail · Mohamed S. Mohamed · Tarek
Ahmed Mahmoud